Algoritmi 1

I seguenti appunti sono a cura di ***Catalin Ceban*** *e si basano sul corso di "Algoritmi 1" erogato nell'A.A. 2026 dalla professoressa Tiziana Calamoneri per il C.d.L Informatica (33503)*

5 novembre 2026

Indice

1 Introduzione **3**

1.1 Costo computazionale e Random Access Machine (RAM) 4

1.2 Notazione asintotica 5

1.3 Ordini di infinito e sommatorie notevoli 5

2 Il problema della ricerca **7**

2.1 Ricerca sequenziale 7

2.2 Ricerca binaria 8

3 Il problema dell'ordinamento **13**

3.1 Algoritmi di ordinamento naif 13

3.1.1 Insertion Sort 13

3.1.2 Selection Sort 16

3.1.3 Bubble Sort 17

3.2 Algoritmi di ordinamento efficienti 19

3.2.1 Complessità del problema dell'ordinamento 19

3.2.2 Merge Sort 20

3.2.3 Quick Sort 24

3.2.4 HeapSort 27

3.3 Ordinamenti lineari 35

3.3.1 Counting Sort 35

3.3.2 Bucket Sort 38

4 Strutture dati **39**

4.1 Array 42

4.2 Lista concatenata semplice 43

4.3 Lista concatenata doppia 46

4.4 Coda (Queue) 48

4.5 Coda con priorità 50
4.6 La Pila (Stack) 51
4.7 Albero 52
4.7.1 Alberi binari 54
4.7.2 Rappresentazione in Memoria degli Alberi 55
4.7.3 Visite degli Alberi 57
4.7.4 Visite per livelli (Breadth-First Search) 60
4.8 Dizionario 61
4.8.1 Tabelle ad indirizzamento diretto 62
4.8.2 Tabelle hash 63
4.8.3 Alberi binari di ricerca (ABR) 63
4.9 Alberi AVL 68
4.9.1 Casistica delle Rotazioni 70
4.9.2 Inserimento e Cancellazione 71
4.10 B-Alberi 72
4.10.1 Rappresentazione in Memoria 73
4.10.2 Operazioni sui B-Alberi 74
4.11 Riepilogo Costi e Casistiche d'Uso 75

1

Introduzione

Definizione 1.0.1: Definizione di algoritmo

Un algoritmo è una sequenza finita di operazioni elementari e univoche, mirate a risolvere un problema, che impiegano un tempo finito e operano su strutture dati. In questa definizione, vediamo alcune parole chiave di questa definizione:

- **elementari**: ogni operazione deve rappresentare una e un'unica istruzione macchina. Un'operazione elementare **non deve** poter essere scomposta in comandi più semplici. Le operazioni elementari sono infatti le **operazioni aritmetiche** (+, -, *, //, /, %), di **confronto** (if/else), lettura di un dato (nel pratico significa **accedere al valore di una variabile**), scrittura su memoria centrale (nel pratico vuol dire **assegnare un valore ad una variabile**) (importante che sia la memoria centrale, siccome quella di massa non è banale come operazione).
- **univoche**: ogni operazione deve essere **interpretata nello stesso modo da tutti**. Si stabiliscono infatti delle convenzioni nell'uso delle operazioni per scrivere algoritmi. In caso di variazioni nella notazione, è estremamente raccomandato riportare delle annotazioni nella scrittura dell'algoritmo che spieghino eventuali operazioni ambigue.
- **tempo finito**: significa che l'algoritmo deve essere eseguito in una **quantità di tempo ragionevole**, non deve andare avanti per sempre.
- **strutture dati**: una struttura dati è un particolare **strumento che permette la memorizzazione dei dati** in un particolare modo e l'accesso a questi, che sono memorizzati in memoria centrale. Esistono varie strutture dati, ognuna delle quali permette l'accesso ai dati in modo diverso. Ognuna

ha i suoi vantaggi e svantaggi: non esiste dunque una struttura dati perfetta per ogni tipo di problema.

1.1 Costo computazionale e Random Access Machine (RAM)

Tornando al discorso del "tempo finito", è giusto parlare del costo computazionale di un algoritmo.

Definizione 1.1.1: Definizione di costo computazionale

Il **costo computazionale** (oppure **complessità computazionale**) di un algoritmo è la misura delle risorse richieste per la sua esecuzione, in funzione della dimensione dell'input (generalmente indicata con n). Questo si divide in:

- **complessità temporale**, cioè il numero di operazioni elementari eseguite dall'algoritmo (nota: non usiamo i secondi come riferimento, in quanto con l'avanzare del tempo gli elaboratori diventano sempre più veloci, mentre il numero di operazioni di un algoritmo rimane tale).
- **complessità spaziale**, che rappresenta la quantità di memoria centrale extra necessaria all'algoritmo per arrivare alla soluzione.

Definizione 1.1.2: Definizione di Rabdom Access Machine (RAM)

Per aiutarci nel calcolo del costo computazionale di un algoritmo, gli informatici teorici hanno sviluppato un modello di elaboratore resistente a qualsiasi epoca, che non si basa dunque sui progressi scientifici, ma che possiede delle caratteristiche ben precisate. Il suo nome è modello RAM (Random Access Machine), e possiede le seguenti caratteristiche:

- è un **mono processore**, che dunque esegue le operazioni in modo **sequenziale**, una dopo l'altra;
- esegue le **operazioni elementari** (quelle definite in 1.0.1) **in tempo costante**;
- opera in **Misura di Costo Uniforme (MCU)**, cioè un paradigma che prevede che i **valori** trattati dall'algoritmo possano essere **memorizzati in una sola word di memoria** (generalmente 32 o 64 bit, dipende dal processore).

Un'alternativa alla MCU è la **Misura di Costo Logaritmico**, cioè un paradigma che opera secondo la supposizione che i **dati** siano **così grandi da non poter essere salvati in una singola word di memoria**. Tuttavia nel corso di queste dispense vedremo soltanto la MCU.

1.2 Notazione asintotica

La **notazione asintotica** è uno **strumento** puramente matematico che utilizziamo per **fornire delle stime riguardo al costo computazionale di un algoritmo**. Fermandoci a riflettere sul nome di questo strumento, notiamo la parola "asintotica": questo termine indica che andiamo a studiare il comportamento del costo *all'infinito*, ovverosia al crescere dell'input (insomma, ci interessa sapere come si comporta l'algoritmo per input grandi, in modo da sapere quale algoritmo usare per input grandi e quale per input piccoli).
Vediamo di seguito le 3 notazioni asintotiche fondamentali per lo studio del costo degli algoritmi:

- O-grande $f(n) = O(g(n))$ Dire che f è un O-grande di g equivale a dire che f è **sempre più piccola** di g, soprattutto asintoticamente. In parole povere, l'O-grande ci fornisce un **limite superiore** per il costo del nostro algoritmo: nel **peggiore** dei casi, l'algoritmo avrà un tempo pari a $g(n)$.
- Ω-grande $(f(n)) = \Omega(g(n))$. Dire che f è un Ω-grande di g equivale a dire che f è sempre più grande di g, soprattutto asintoticamente. Al contrario dell'O-grande, che stabiliva un limite superiore, l'Ω-grande ci fornisce un **limite inferiore** per il nostro algoritmo: ci dice che non andrà **mai più lentamente** di una certa funzione $g(n)$.
- Θ-grande $(f(n) = \Theta(n))$ Dire che f è un Θ-grande di g equivale a dire che la funzione f e la funzione g, almeno asintoticamente, si comportano **allo stesso modo**. Il Θ-grande ci fornisce un limite stretto per l'andamento asintotico del nostro algoritmo, dicendo che ci metterà esattamente quel tempo per essere eseguito.

1.3 Ordini di infinito e sommatorie notevoli

Nello studio del costo computazionale degli algoritmi, assieme alle notazioni asintotiche e alla loro algebra, è utile conoscere la gerarchia delle funzioni all'infinito e alcune sommatorie notevoli utili nei calcoli:

$$1 << \log^k(n) << n^{\frac{1}{k}} << n << n \cdot \log^k(n) << n^k << a^n << n! << n^n$$

Sommatoria di Gauss: $\sum_{i=0}^{n} i = \Theta(n^2)$

Sommatoria esponenziale: $\sum_{i=0}^{n} 2^i = \Theta(2^n)$

Sommatoria logaritmica: $\sum_{i=0}^{n} \log(i) = \Theta(n \cdot \log(n))$

2

Il problema della ricerca

Nel mondo dell'informatica sono 2 i problemi più ricorrenti: ricerca di un valore in una sequenza e ordinamento di una sequenza. In questo paragrafo ci occuperemo di trattare il primo di questi problemi. Vediamo di seguito i due algoritmi principali che si occupano della ricerca.

2.1 Ricerca sequenziale

Come suggerisce il nome, questo algoritmo cerca un valore nel modo più umano e naturale possibile: scorre tutti i valori della sequenza e, se lo trova, restituisce l'indice dell'elemento all'interno della sequenza; se non lo trova, restituisce -1 oppure None, a seconda delle implementazioni. Il codice associato a questo algoritmo è il seguente:

```
1       def ricerca_sequenziale(A, target):
2         n = len(A)
3         for i in range(n):
4           if A[i] == target:
5             return i
6         return -1
```

Listing 2.1: Codice della ricerca sequenziale

Spiegazione algoritmo 2.1.1: Ricerca sequenziale

L'algoritmo appena presentato risulta molto semplice ed intuitivo:

1. **A** è l'array, la sequenza di dati dove dobbiamo cercare il valore;
2. **target** è il valore che stiamo cercando; Scorriamo tutto l'array, a partire

dall'elemento ad indice 0 fino a quello con indice n-1.

3. Se troviamo il valore, allora restituiamo subito l'indice dell'elemento, facendo finire l'esecuzione del programma.

4. Se finito il ciclo non siamo usciti anticipatamente, vuol dire che il valore non era presente, e dunque restituiamo -1 (oppure un qualsiasi valore che segnali l'assenza del target nella sequenza).

Visto l'algoritmo, procediamo con lo scriverne il costo computazionale:

- **Caso migliore:** $\Theta(1)$, il target si trova al primo indice della sequenza.
- **Caso peggiore:** $\Theta(n)$, il target **non** è presente, oppure si trova all'ultimo elemento della sequenza, e per trovarlo sarà dunque necessario scorrerla tutta.

Questo algoritmo, come detto prima, è abbastanza universale: può essere eseguito su qualunque struttura dati lineare in maniera analoga.

2.2 Ricerca binaria

Con questo algoritmo si inizia già ad entrare più nel fascino della materia: vedere le strategie adottate dagli analisti degli algoritmi per risolvere un problema, e in particolare per ottimizzarlo, cioè per migliorarne il costo computazionale. Questo algoritmo prende esempio da come cercheremmo le parole all'interno di un dizionario:

- Apriamo a metà libro e controlliamo su che lettera ci troviamo.
- Se la lettera è più grande, allora cerchiamo nella metà sinistra.
- Se la lettera è più piccola, allora cerchiamo nella metà destra.
- Ripetiamo in questo modo finché non abbiamo trovato la parola che cercavamo.

Come vediamo però, questo algoritmo parte da una supposizione molto forte: che la sequenza sia **ordinata**. Discuteremo del problema dell'ordinamento in seguito, ma è giusto precisare che anche quegli algoritmi hanno un loro costo computazionale.

Dunque sorge spontanea una domanda: qual è il vantaggio nell'uso di questo algoritmo, se so che devo usarlo su una sequenza già ordinata? Supponiamo di star lavorando su grandi quantità di dati: se dovessimo fare una ricerca su una sequenza **una sola volta**, allora certo, conviene usare la ricerca sequenziale: evitiamo di ordinare la sequenza e

troviamo subito l'elemento. Ma quando dobbiamo cercare un valore già due, tre, quattro, ..., volte, insomma, ci rendiamo conto che forse ottimizzare la ricerca non sarebbe una brutta idea. Se si tratta di un'operazione che facciamo spesso, forse vale effettivamente la pena ordinare quella sequenza.

Fatta la dovuta ipotesi sull'ordinamento, e risposta la domanda, possiamo procedere col mostrare il codice dell'algoritmo. Questo algoritmo può essere scritto in una sua versione iterativa e ricorsiva, che riporteremo entrambe per completezza.

```
def ricerca_binaria(A, target):
  n = len(A)
  inizio, fine = 0, n - 1

  while (inizio <= fine):
    mid = (inizio + fine) // 2
    if A[mid] == target:
      return mid
    elif A[mid] < target:
      inizio = mid + 1
    else:
      fine = mid - 1

  return -1
```

Listing 2.2: Versione iterativa della ricerca binaria

Spiegazione algoritmo 2.2.1: Ricerca binaria iterativa

1. I parametri presi in input sono l'array A, che contiene i valori in cui cercare, e target, che contiene l'elemento che stiamo cercando.

2. Come prima cosa, ci calcoliamo i valori di inizio e fine per la parte di sequenza che stiamo considerando (ricordiamo l'esempio del dizionario, ogni volta ci spostiamo a destra e a sinistra a seconda del risultato del confronto). Questi saranno inizio e fine, che inizialmente saranno 0 (inizio dell'array) e len-1 (ossia l'ultimo indice disponibile).

3. Impostiamo un ciclo while. Potremmo mettere varie condizioni, ma decidiamo di utilizzare inizio $\leq$ fine. Questo sarà chiaro nei prossimi punti di spiegazione, ma per ora ci limitiamo a dire che man mano dovremo incrementare inizio e

decrementare fine, quindi quando avremo inizio > fine, cioè si saranno scavallati, la condizione del while risulterà falsa e usciremo dal ciclo senza aver trovato il valore.

4. Ci calcoliamo il valore di mezzo (mid) della sequenza che stiamo considerando. Pensiamo a questo come ad "aprire a metà del dizionario".

5. Adesso dobbiamo effettuare dei controlli:

   (a) Se il valore all'indice mid è UGUALE A che stiamo cercando, allora siamo stati fortunati e possiamo uscire dal ciclo, restituendo proprio mid, che è l'indice al quale si trova il valore.

   (b) Se il valore all'indice mid è MINORE di quello che stiamo cercando, dato che l'array è ordinato, significa che dovremo cercare nella metà destra dell'array. Proceduralmente questo si traduce incrementando il valore di inizio (ricordiamo che inizio è l'indice che rappresenta l'inizio della sequenza che stiamo considerando). Impostiamo il nuovo indice di inizio a mid + 1 (chiaramente non a mid, dato che il valore nell'indice mid l'abbiamo già considerato nella prima condizione).

   (c) Se il valore all'indice mid è maggiore di quello che stiamo cercando, dato che l'array è ordinato, significa che dovremo cercare nella metà sinistra dell'array. Proceduralmente questo si traduce decrementando il valore di fine (ricordiamo che fine è l'indice che rappresenta la fine della sequenza che stiamo considerando). Impostiamo il nuovo indice di fine a mid -1 (chiaramente non a mid, dato che il valore nell'indice mid l'abbiamo già considerato nella prima condizione).

6. Avendo fatto i dovuti controlli ed incrementi, il ciclo procederà finché il valore non verrà trovato o finché gli indici si scavalleranno: in tal caso si restituisce -1 in quanto significa che il valore non è stato trovato.

```
1       def ricerca_binaria(A, target, inizio, fine):
2         if inizio > fine:
3           return -1
4
5         mid = (inizio + fine) // 2
6         if A[mid] == target:
7           return mid
```

```
8           elif A[mid] < target:
9             return ricerca_binaria(A, target, mid + 1, fine)
10          else:
11            return ricerca_binaria(A, target, inizio, mid - 1)
```

Listing 2.3: Versione ricorsiva della ricerca binaria

Spiegazione algoritmo 2.2.2: Ricerca binaria ricorsiva

1. I parametri presi in input sono l'array A, che contiene i valori in cui cercare; target, che contiene l'elemento che stiamo cercando; inizio, che rappresenta l'indice di *inizio della sequenza* che stiamo considerando; fine, che rappresenta l'indice di *fine della sequenza* che stiamo considerando.

2. Come prima cosa, essendo un algoritmo ricorsivo, settiamo il caso base, che sarà nient'altro che la condizione di *uscita dal ciclo* della versione iterativa: inizio $>$ fine, cioè si scavallano.

3. Ci calcoliamo poi il valore dell'indice centrale mid (nuovamente, la pagina "centrale" del dizionario).

4. Qui procediamo analogamente, facendo i nostri confronti:

   (a) Se il valore che cerchiamo è uguale all'elemento in indice mid, allora possiamo restituire l'indice stesso.

   (b) Se l'elemento in indice mid è più piccolo del valore che cerchiamo, ci spostiamo nella parte destra: ricorsivamente, questo vuol dire fare una nuova chiamata alla funzione, assicurandoci però di cambiarne i parametri (altrimenti avremmo una ricorsione infinita!). In questo caso il cambio di parametri è semplicemente un ricalcolo del valore di inizio, passato prima come secondo parametro, che stavolta viene sostituito da mid + 1.

   (c) Se l'elemento in indice mid è più grande del valore che cerchiamo, ci spostiamo nella parte sinistra. In questo caso il cambio di parametri è semplicemente un ricalcolo del valore di fine, passato prima come terzo parametro, che stavolta viene sostituito da mid - 1.

Analizzati i codici relativi agli algoritmi, andiamo a verificarne il costo computazionale, nella sua versione iterativa e ricorsiva:

- **Caso migliore:** $\Theta(1)$, troviamo l'elemento alla prima iterazione/ricorsione, vale a dire all'indice di mezzo mid.
- **Caso peggiore:** $\Theta(\log(n))$, troviamo l'elemento all'ultima iterazione/ricorsione oppure non lo troviamo proprio.

Visti i due principali algoritmi di ricerca, di seguito viene lasciata una tabella di confronto tra questi.

Algoritmo	**Caso migliore**	**Caso peggiore**
Ricerca sequenziale	$\Theta(1)$	$\Theta(n)$
Ricerca binaria	$\Theta(1)$	$\Theta(\log(n))$

3

Il problema dell'ordinamento

Assieme alla ricerca di un valore in un insieme di elementi, il problema dell'ordinamento è l'altro grande pilastro dello studio degli algoritmi. Come si può intuire dal nome di questa gamma di problemi, lo scopo di questi è di ordinare una sequenza data in input, in base ad una relazione di ordine che può essere applicata sull'insieme stesso. Un esempio di relazione d'ordine può essere quella di maggiore/minore tra due numeri. Chiaramente questa relazione d'ordine deve essere definita sull'insieme dei valori da ordinare: non possiamo ordinare delle persone con lo stesso criterio con cui ordiniamo dei numeri: per stabilire chi sia maggiore o minore tra due persone, ad esempio, dobbiamo decidere quale è il criterio per cui le confrontiamo (es: l'età, l'altezza, ...) e solo allora potremmo effettivamente procedere con l'ordinamento.

Nel corso della storia degli algoritmi sono stati proposti vari algoritmi di ordinamento, partendo da quelli più "intuitivi" ma spesso meno efficienti, per arrivare poi ad alcuni meno "intuitivi" ma decisamente più efficienti.

3.1 Algoritmi di ordinamento naif

3.1.1 Insertion Sort

Questo algoritmo ordina gli elementi dell'insieme come ordineremmo le carte di un mazzo che sono nella nostra mano: prendiamo una carta e, se si trova nel posto sbagliato, cioè non rispetta l'ordinamento rispetto alle carte precedenti/successive, la mettiamo nel posto giusto. Intuitivamente questo accade "liberando uno spazio" dove dobbiamo posizionare la carta e poi facendo "scalare a destra" tutte quelle che sono rimaste. Proponiamo di seguito il codice dell'algoritmo:

```
1       def insertion_sort(A):
```

```
2         n = len(A)
3         for i in range(1, n):
4           chiave = A[i]
5           j = i - 1
6
7           while (j >= 0) and (chiave < A[j]):
8             A[j+1] = A[j]
9             j -= 1
10
11          A[j+1] = chiave
12
13        return A
```

Listing 3.1: Insertion Sort

Commentiamone adesso le istruzioni:

Spiegazione algoritmo 3.1.1: Insertion Sort

1. Apriamo un ciclo for che scorre tutti gli elementi dell'array: essenziale per *ogni* algoritmo di confronto, perché comunque bene o male bisogna scorrere tutti gli elementi per capire se è ordinato o meno. Partiamo dal secondo elemento perché il primo lo consideriamo già "ordinato".

2. Per ogni iterazione, consideriamo l'elemento corrispondente al corrente indice di iterazione (sarebbe "chiave"). Sarebbe come la "nuova carta" che abbiamo appena pescato, dobbiamo vedere come si relaziona quella con tutte quelle prima.

3. Adesso dobbiamo dunque scorrere all'indietro: prendiamo un indice j che facciamo partire dall'elemento prima della chiave, in modo che ci aiuti a scorrere all'indietro per vedere come si relaziona la chiave con gli elementi che stanno prima di lei.

4. Apriamo un ciclo while per scorrere l'array all'indietro. La condizione del ciclo sarà doppia:

   (a) j >= 0 perché vogliamo evitare che l'indice che va al contrario vada fuori dall'array.

   (b) chiave < A[j], perché dobbiamo assicurarci l'elemento che stiamo considerando NON rispetti la relazione d'ordine secondo la quale vogliamo

ordinare (se vogliamo ordinare in ordine crescente, mettiamo $<$, mentre decrescente avremmo messo $>$). Questo perché se la chiave non va scambiata con l'elemento precedente, significa che non vanno controllati neanche quelli di prima dato che erano già ordinati. In tal caso non si entrerebbe proprio nel ciclo interno, il che è un ottimo controllo per migliorare il costo computazionale dell'algoritmo.

5. Se si entra nel ciclo while, significa che dobbiamo effettivamente effettuare lo "slittamento" degli elementi. **ATTENZIONE**: Non è uno scambio, è uno slittamento, quindi è proprio per questo che facciamo A[j+1] = A[j]. Se siamo entrati nel ciclo significa che dobbiamo creare spazio a sinistra, dunque tutti gli elementi di sinistra (j) li mettiamo di uno più a destra (j+1).

6. Decrementiamo l'indice j per fargli scorrere anche gli elementi precedenti e controllare come si relazione.

7. Qui è una piccola accortezza: siccome ogni ciclo while decrementa j di 1, se all'ultimo ciclo j fosse stato 0 otterremmo, in uscita dal ciclo, j = -1. Questo NON è un indice valido, e dunque vuol dire che dobbiamo rimettere la chiave all'elemento j+1, perché è l'ultimo che ha "lasciato spazio" insieme agli altri elementi a destra.

Visto il codice, possiamo definire il costo computazionale nel caso migliore e peggiore:

- **Caso migliore: $\Theta(n)$**, e si verifica quando l'array è già ordinato. Come anticipato nell'analisi del codice, questo è possibile grazie al fatto che il ciclo while è provvisto di una condizione che verifica se si devono *effettivamente* slittare gli elementi oppure no.

- **Caso peggiore: $\Theta(n^2)$**, e si verifica quando l'array è ordinato al contrario rispetto alla relazione d'ordine che abbiamo stabilito. In quel caso il calcolatore entrerà nel ciclo while tante volte quante sono gli elementi dell'array che rimangono, cioè 1 volta alla prima iterazione, 2 volte alla seconda, 3 volte alla terza,..., n volte alla n-esima.

Punto forte di questo algoritmo, il che non è scontato è che **ordina in loco**, cioè ordina l'array su se stesso, senza utilizzare ulteriore memoria per array d'appoggio. Inoltre, l'Insertion Sort è un algoritmo **stabile**, il che vuol dire che se vengono trovati due elementi la cui chiave di ordinamento è uguale, nella sequenza finale ordinata verranno mantenuti nell'ordine in cui apparivano nella sequenza disordinata.

3.1.2 Selection Sort

Questo algoritmo è anche più intuitivo del precedente: ad ogni iterazione cerchiamo il minimo (o il massimo, dipende da se vogliamo operare in ordine crescente o decrescente) nella parte restante dell'array e lo scambiamo con l'elemento corrente che stiamo considerando. Molto semplice ed intuitivo. Questo algoritmo richiede però una funzione di supporto chiamata "trova_minimo(A, indice_inizio)" che, come suggerisce il nome, ci aiuta a trovare il minimo nella parte di array che ci manca.
Vediamo ora il codice di questo algoritmo:

```
def trova_minimo(A, indice_inizio):
  n = len(A)
  indice_min = A[indice_inizio]
  for i in range(indice_inizio + 1, n):
    if A[i] < A[indice_min]:
      indice_min = i
  return indice_min
```

Listing 3.2: Funzione ausiliaria "trova_minimo"

```
def selection_sort(A):
  n = len(A)

  for i in range(n):
    indice_min = trova_minimo(A, i)
    A[i], A[indice_min] = A[indice_min], A[i]

  return A
```

Listing 3.3: Selection Sort

Di seguito viene analizzato il codice dell'algoritmo di ordinamento (viene omessa l'analisi della funzione di ricerca del minimo in quanto considerata piuttosto banale).

Spiegazione algoritmo 3.1.2: Selection Sort

1. Apriamo un ciclo for che scorre tutto l'array.
2. Chiamiamo la funzione trova_minimo(array, i) che ci restituisce l'indice del-

l'elemento più piccolo nel sottoarray che va dall'indice corrente (escluso) ad n, cioè la fine dell'array.

3. Una volta trovato l'indice dell'elemento minimo, procediamo allo scambio, scambiando l'elemento all'indice dell'iterazione corrente i con l'elemento minimo del sottoarray di destra.

4. Restituiamo l'array ordinato.

Anche questo algoritmo ordina **in loco**. In questo caso il costo computazionale dell'algoritmo sarà **equivalente nel Caso migliore e nel Caso peggiore**, in quanto non sono presenti condizioni che impediscano ad alcuni blocchi di codice di essere eseguiti, bensì vengono eseguite n iterazioni del ciclo for esterno e per ogni iterazione si cerca il minimo negli n-i elementi rimanenti dell'array. Il costo computazionale sarà dunque $\mathbf{\Theta(n^2)}$ in entrambi i casi. A sua volta, il Selection Sort **NON è un algoritmo stabile**, in quanto se per esempio avessimo un array di tutti numeri uguali, l'ultimo numero verrebbe messo al primo posto (ad esempio alla prima iterazione), e in generale verrebbero tutti ugualmente scambiati.

3.1.3 Bubble Sort

Anche questo risulta abbastanza intuitivo: il concetto di base è di confrontare ogni coppia di elementi, e se questa non rispetta la relazione di ordine scelta, scambiarli. Senza perderci in ulteriori chiacchere, procediamo con il codice dell'algoritmo:

```
1       def bubble_sort(A):
2         n = len(A)
3
4         for i in range(n):
5           for j in range(n - i - 1):
6             if (A[j] > A[j+1]):
7               A[j], A[j+1] = A[j+1], A[j]
8
9         return A
```

Listing 3.4: Bubble Sort

Spiegazione algoritmo 3.1.3: Bubble Sort

1. Ciclo for che itera su tutti gli elementi dell'array.
2. Questo ciclo si ripete per un numero sempre minore di volte. Questo perché alla prima iterazione, se continuiamo a scambiare elementi due a due, l'elemento **massimo** sarà in fondo all'array. Potremo dunque, dopo la prima iterazione, controllare un elemento in meno. Dopo la seconda avremo il **secondo massimo** al penultimo posto, dunque potremo controllare poi un elemento in meno. E così via...
3. Qui controlliamo se due elementi adiacenti nel ciclo interno NON rispettano la relazione d'ordine: in tal caso dovremo scambiarli. Ci chiediamo, perché non possiamo fare questo nel ciclo esterno, utilizzandone solo uno? Beh, perché se scambiassimo solo una volta a 2 a 2, avremo che sicuramente il massimo si trova in ultima posizione (come anticipato prima) ma degli altri elementi non avremo garanzia!

Il costo computazionale di questo algoritmo è, in entrambi i casi, $\mathbf{\Theta(n^2)}$, in quanto abbiamo due cicli for nidificati. Anche il Bubble Sort ordina **in loco**, ed è anche un algoritmo **stabile**.

Il Bubble Sort, sopra descritto, è l'ultimo dei tre algoritmi di ordinamento "ingenui". Per riassumere questi algoritmi semplici (ma non sempre efficienti) si lascia una tabella riassuntiva con costi computazionali, vantaggi e svantaggi, casi d'uso di questi vari algoritmi.

Algoritmo	**Costo computazionale**	**Stabile?**
Insertion Sort	Best Case: $\Theta(n)$; Worst Case: $\Theta(n^2)$	Si
Selection Sort	Best Case, Worst Case: $\Theta(n^2)$	No
Bubble Sort	Best Case, Worst Case: $\Theta(n^2)$	Si

Fatti i dovuti riassunti in base alla stabilità e al costo computazionale di questi algoritmi, forniamo di seguito alcuni casi d'uso:

- Insertion Sort: ottimo per array quasi ordinati, pessimo per array ordinati al contrario.

- Selection Sort: generalmente quasi mai usato, ma potrebbe essere una buona soluzione quando si lavora su dispositivi in cui il costo di scrittura è molto più elevato di quello di lettura.
- Bubble Sort: generalmente sconsigliato se implementato nella sua versione classica. Se viene invece implementato con un flag che tiene traccia degli scambi effettuati, il costo nel caso migliore può piombare ad un fantastico $\Theta(n)$.

3.2 Algoritmi di ordinamento efficienti

3.2.1 Complessità del problema dell'ordinamento

Prima di passare ad algoritmi più efficienti rispetto a quelli appena visti, si fa una digressione riguardante la natura intrinseca del problema dell'ordinamento. Essendo un problema così comune, ed essendo costantemente alla ricerca di algoritmi più efficienti, ci si è chiesto: ma quanto possono essere efficienti questi algoritmi? C'è una specie di *limite inferiore* sotto al quale gli algoritmi di ordinamento NON possono andare? Cioè che tutti gli algoritmi di ordinamento (ATTENZIONE: **quelli basati sui confronti**) hanno un costo minimo pari ad una certa funzione?

Si è riusciti a stabilire un limite inferiore per gli algoritmi basati sui confronti, mediante osservazioni matematiche riguardanti la natura dell'oggetto da ordinare. Inoltre, per giungere alla conclusione sperata, si è anche utilizzata una struttura chiamata "albero decisionale". Questa struttura permette di rappresentare tutte le possibili permutazioni di una sequenza di n elementi, che sappiamo essere $n!$. L'obiettivo di un algoritmo di ordinamento è trovare l'unica permutazione giusta tra le $n!$ totali.

Dal nome "albero decisionale" possiamo intuire come la struttura sia quella di un albero, cioè dove abbiamo una radice, dei nodi e delle foglie. Nell'ottica dell'albero decisionale, abbiamo che:

- La radice è la sequenza di elementi così come ci viene data in partenza.
- Ogni ramo rappresenta un confronto tra due elementi.
- Ogni nodo è l'esito di un confronto operato da un ramo, che da dunque origine ad una nuova disposizione degli elementi di partenza.
- Le foglie rappresentano le permutazioni finali, quelle ordinate.

Siccome l'algoritmo di ordinamento deve ordinare, il numero di foglie (cioè di permutazioni finali ordinate!!!) deve essere *almeno* $n!$. Qui entra in gioco un accorgimento sulle

strutture ad alberi: per andare dalla radice ad una foglia, il tempo richiesto è proporzionato all'altezza dell'albero (che indichiamo con h). In un albero binario di altezza h, il numero di foglie può essere al massimo 2^h (perché per ogni livello si raddoppia il numero dei nodi).

Dunque, sappiamo che il numero massimo dei nodi di un albero binario è 2^h. Sappiamo anche che il numero minimo di permutazioni (ricordiamo, le foglie dell'albero decisionale) deve essere almeno $n!$. Impostiamo dunque la disequazione $2^h \geq n!$. Isolando h, otteniamo che $h \geq \log(n!)$, che possiamo approssimare, grazie alla formula di Stirling, come $n \cdot \log(n)$.

Siccome otteniamo che $h \geq n \cdot \log(n)$, abbiamo ottenuto che $h = \Omega(n \cdot \log(n))$. Dunque OGNI algoritmo di ordinamento basato sui confronti ha un costo *almeno pari* ad $n \cdot \log(n)$.

Di conseguenza, per tutti gli algoritmi che vedremo di seguito, per il costo temporale ci basterà che questo sia un $\Theta(n \cdot \log(n))$ (perché sarebbe letteralmente il meglio a cui possiamo aspirare). Dovremo dunque stare particolarmente attenti al costo spaziale, dunque facendo in modo che l'algoritmo non usi spazio ulteriore in memoria, ma ordini in loco (quindi modifichi l'array stesso che passiamo come input).

3.2.2 Merge Sort

L'algoritmo *Merge Sort* è un algoritmo inerentemente ricorsivo che adotta l'importantissima tecnica del *"divide et impera"* (*"divide and conquer"* in inglese), che descrive perfettamente la natura di un problema ricorsivo. Si sviscera nei seguenti passaggi:

Divide: il problema "grande" si suddivide in problemi più piccoli, più facili da risolvere;

Impera: i sottoproblemi si risolvono ricorsivamente;

Combina: si mettono insieme le soluzioni dei sottoproblemi per creare la soluzione al problema originale.

Secondo questa logica, il *Merge Sort* opera come segue:

- Divide l'array di n elementi in due sequenze da $n/2$ elementi ciascuna;
- Ordina ricorsivamente le due sequenze di $n/2$ elementi (applicando ciascuna l'algoritmo di *Merge Sort*, per questo è ricorsivo);
- Combina le due sottosequenze di $n/2$ elementi in modo che costruiscano poi la sequenza di n elementi ormai ordinata.

Detto questo, vediamo il codice relativo a questo algoritmo. Nota che dovremo usare una funzione di appoggio chiamata `fondi()` che prende in input due sottosequenze e le combina rendendole una sequenza ordinata.

```
def fondi(A, indiceInizio, indiceMedio, indiceFine):
  i = indiceInizio
  j = indiceMedio + 1
  B = []

  while ((i <= indiceMedio) && (j <= indiceFine)):
    if (A[i] < A[j]):
      B.append(A[i])
      i++
    else:
      B.append(A[j])
      j++

  while (i <= indiceMedio):
    B.append(A[i])
    i++

  while (j <= indiceFine):
    B.append(A[j])
    j++

  for k in range(B.length):
    A[indiceInizio + k] = B[k]
```

Listing 3.5: Funzione di appoggio per la fusione

```
def merge_sort(A, indice_inizio, indice_fine):
  if indice_inizio < indice_fine:
    mid = (indice_inizio + indice_fine) // 2
    merge_sort(A, indice_inizio, mid)
    merge_sort(A, mid+1, indice_fine)
    fondi(A, indice_inizio, mid, indice_fine)
```

Listing 3.6: Merge Sort

Spiegazione algoritmo 3.2.1: Merge Sort

`fondi()` prende come parametri:

- `A`: l'array originale che vogliamo ordinare;
- `indiceInizio`: l'indice di inizio della "sequenza di sinistra" che vogliamo fondere;
- `indiceMedio`: l'indice a metà tra due sequenze, che è anche l'indice di FINE DELLA SEQUENZA DI SINISTRA (incluso, quindi la sequenza di sinistra va da [`indiceInizio`, `indiceMedio`]) e l'indice di INIZIO DELLA SEQUENZA DI DESTRA (escluso, quindi la sequenza di destra va da [`indiceMedio+1`, `indiceFine`]);
- `indiceFine`: l'indice di fine della "sequenza di destra" che vogliamo fondere.

Inizializziamo due indici, ognuno dei quali ci servirà per scorrere una delle due sottosequenze:

- `i`, inizializzato a `indiceInizio`, ci servirà a scorrere la sequenza di sinistra (che ricordiamo appunto iniziare ad `indiceInizio`);
- `j`, inizializzato a `indiceMedio+1`, ci servirà a scorrere la sequenza di destra (che ricordiamo appunto iniziare ad `indiceMedio+1`).

Inizializziamo un array di appoggio che chiamiamo `B`, che ci servirà per inserire progressivamente gli elementi di una delle due sottosequenze (in particolare il minore). Creiamo un array di appoggio proprio perché se andassimo ad operare sull'array originale avremmo una perdita di informazioni: da qui il fatto che il *Merge Sort* NON ordina in loco.
Facciamo partire un ciclo `while` che va avanti finché uno dei due indici che scorre la rispettiva sottosequenza non arriva al suo limite: il limite per `i` è `indiceMedio`, mentre il limite per `j` è `indiceFine`. Abbiamo messo un `&&` tra i due apposta per fare in modo che appena una delle due condizioni è falsa si esca dal ciclo.
Dentro al ciclo confrontiamo i primi due elementi delle rispettive sottosequenze: possiamo farlo dato che, per ipotesi induttiva, le due sottosequenze sono di per sé già ordinate, dunque il primo elemento a sinistra sarà sempre il più piccolo per ciascuna sottosequenza. Se risulta che `A[i]` $<$ `A[j]`, allora mettiamo `A[i]` in fondo all'array di appoggio `B`. A quel punto, dato che abbiamo già controllato l'elemento in posizione `i` della sottosequenza di sinistra, dobbiamo incrementare l'indice `i` per

farlo andare all'elemento successivo. Il procedimento è analogo per quanto riguarda `A[j]`, se fosse il più piccolo.
Come detto precedentemente, il ciclo si ferma quando uno dei due indici raggiunge il suo valore limite. A questo punto impostiamo due cicli `while`, ma SOLO UNO DEI DUE VERRÀ ESEGUITO. Questo perché il codice esce dal ciclo `while` solo quando UNA DELLE DUE sottosequenze sarà terminata. A quel punto non ci resterà che aggiungere in coda tutti gli elementi della sequenza alla quale rimangono ancora degli elementi: se `i` $\leq$ `indiceMedio`, significa che la sottosequenza di sinistra presenta ancora alcuni elementi da mettere in `B`, e dunque possiamo metterli alla fine, tanto siamo sicuri che siano ordinati. Analogamente se vale che `j` $\leq$ `indiceFine`.
A questo punto dobbiamo modificare l'array di partenza che ci è stato fornito, in modo da poterlo restituire effettivamente ordinato. Il ciclo `for` si occupa di copiare ogni singolo elemento di `B` all'interno di `A`. Ma non all'interno dell'INTERO array `A`, bensì deve copiare gli elementi della sottosequenza che va da `indiceInizio` ad `indiceFine`, che sono proprio gli elementi contenuti all'interno dell'array `B`. Questo perché `B` prende gli elementi della sottosequenza di sinistra, che va da `indiceInizio` a `indiceMedio`, e quelli della sottosequenza a destra, che va da `indiceMedio+1` a `indiceFine`. Dunque la lunghezza di `B` sarà proprio la stessa lunghezza delle due sottosequenze che abbiamo passato alla funzione `fondi()`.
Per quanto riguarda la funzione `mergeSort()`:

- Controlliamo se l'indice di inizio è minore di quello di fine: in tal caso procediamo con il passo ricorsivo. Se così non fosse, significa che avremo un array di un solo elemento, che per definizione è già ordinato.
- Ci calcoliamo l'indice medio, per suddividere l'array `A` in due sottosequenze.
- **PASSO RICORSIVO:** Chiamiamo la funzione `mergeSort()` sul sottoarray di sinistra, che va da `indiceInizio` a `indiceMedio`, e poi sul sottoarray di destra, che va da `indiceMedio+1` a `indiceFine`. A quel punto, una volta che la funzione avrà raggiunto la fine delle chiamate ricorsive, possiamo chiamare la funzione `fondi()` passando come parametri l'array `A` di partenza e gli indici relativi alle due sottosequenze.

Analizzato l'algoritmo, possiamo ora dire che il costo computazionale è dato dall'equazione di ricorrenza: $T(n) = 2T(n/2) + \Theta(n)$, dove $T(n/2)$ sono le chiamate ricorsive sulla sottosequenza sinistra e destra, mentre $\Theta(n)$ è il costo della funzione `fondi()` (che chiaramente è lineare in quanto dovrà andarsi a scorrere tutti gli elementi del sottoarray di sinistra e di quello di destra). Il risultato di quest'equazione è proprio un $\Theta(n$

$logn$), che è dunque il meglio che possiamo ottenere da un algoritmo di ordinamento basato sui confronti.

Dunque... basta così? Abbiamo già trovato l'algoritmo migliore di tutti? Purtroppo no, in quanto il *Merge Sort* NON ordina in loco: ha infatti un costo spaziale pari a $\Theta(n)$, e questo è un grandissimo svantaggio quando si opera con grandi quantità di dati.

Dunque, andiamo adesso a vedere un altro algoritmo di ordinamento, il cosiddetto *Quick Sort*.

3.2.3 Quick Sort

L'algoritmo *Quick Sort*, nonostante nel caso peggiore abbia un costo computazionale pari a $\mathcal{O}(n^2)$, è spesso una scelta preferibile rispetto al *Merge Sort*, per due motivazioni principali:

- Il costo computazionale nel caso peggiore non viene raggiunto quasi mai, infatti il suo costo medio è un $\Theta(n$
  $logn)$;
- Ordina **in loco**, il che lo rende decisamente molto meglio rispetto ad un *Merge Sort*.

Anche il *Quick Sort* è un algoritmo ricorsivo che usa la tecnica del *"divide et impera"*, anche se in modo diverso dal *Merge Sort*. Opera seguendo questi passaggi:

Divide: Seleziona un elemento "speciale", chiamato pivot, dalla sequenza di n elementi fornita in input. La sequenza originale viene dunque divisa in due sottosequenze, che avranno come centro il pivot: la sottosequenza di sinistra avrà gli elementi minori o uguali del pivot, mentre quella di destra avrà quelli maggiori o uguali.

Impera: Le due sottosequenze vengono ordinate ricorsivamente.

Combina: Non occorre, in quanto ordina in loco.

Esistono varie implementazioni del *Quick Sort*, che variano in base alla scelta del pivot. Nell'implementazione proposta di seguito si sceglie il pivot come l'ultimo elemento di ogni sequenza. Come prima cosa andremo ad implementare la funzione `partiziona()`, che si occupa di ordinare gli elementi come detto sopra: pivot al centro, più piccoli a sinistra e più grandi a destra.

```
1       def partiziona(A, indice_inizio, indice_fine):
2         pivot = A[indice_fine]
```

```
3         i, j = indice_inizio, indice_fine
4
5         while true:
6           while A[j] > pivot:
7             j -= 1
8           while A[i] < pivot:
9             i += 1
10          if i < j:
11            A[i], A[j] = A[j], A[i]
12          else:
13            return j
```

Listing 3.7: Funzione ausiliaria "partiziona"

```
1       def quick_sort(A, indice_inizio, indice_fine):
2         if indice_inizio < indice_fine:
3         indice_pivot = partiziona(A, indice_inizio, indice_fine
             )
4         quick_sort(A, indice_inizio, indice_pivot-1)
5         quick_sort(A, indice_pivot+1, indice_fine)
```

Listing 3.8: Quick Sort

Spiegazione algoritmo 3.2.2: Quick Sort

Nella funzione `partiziona()` abbiamo due indici:

- `i`, che scorre da sinistra a destra e continua finché tutti gli elementi trovati sono validi, cioè stanno nel posto in cui dovrebbero stare. Questi sono gli elementi MINORI DEL PIVOT, che infatti dovrebbero stare nella parte sinistra. Il ciclo si ferma infatti quando viene trovato un elemento maggiore/uguale al pivot, che infatti NON dovrebbe stare in quella porzione di array.
- `j`, che scorre da destra a sinistra e continua finché tutti gli elementi trovati sono validi, cioè stanno nel posto in cui dovrebbero stare. Questi sono gli elementi MAGGIORI DEL PIVOT, che infatti dovrebbero stare nella parte destra. Il ciclo si ferma infatti quando viene trovato un elemento minore/uguale al pivot, che infatti NON dovrebbe stare in quella porzione di array.

Alla fine di entrambe le ricerche si controlla se `i` < `j`: significa che i due indici NON si sono scavallati, e dato che uno indica i piccoli e l'altro i grandi rispettivamente, dobbiamo scambiarne gli elementi a cui puntano. Tutto questo discorso si ripete finché non abbiamo `i` == `j` oppure `i` > `j`, il che vuol dire che gli indici COINCIDONO oppure SI SONO SCAVALLATI, dunque vuol dire che ormai la sottosequenza è correttamente ordinata. A questo punto possiamo restituire `j`, ma è indifferente quale sia.
Il codice della funzione `quickSort()`, invece, è molto più semplice:

- Controlliamo se `indiceInizio` sia minore di `indiceFine`: in tal caso procediamo. Se così non fosse, significherebbe che avremmo una sequenza di un solo elemento, che per definizione è già ordinata.
- Andiamo a chiamare la funzione `partiziona()` sulla sequenza limitata da `indiceInizio` e `indiceFine`, in modo da ottenere l'indice del pivot.
- A questo punto dobbiamo chiamare ricorsivamente la funzione `quickSort()` sulla sequenza di sinistra (che va da `indiceInizio` a `indicePivot` -1) e sulla sequenza di destra (che va da `indicePivot` $+1$ a `indiceFine`). Nota che non includiamo il pivot in nessuna delle due sequenze, proprio perché è quell'elemento speciale che abbiamo scelto e di cui non vogliamo cambiare l'indice, in quanto per definizione è l'unico elemento già ordinato della sequenza.

Il costo computazionale è stato discusso già precedentemente: nel caso peggiore è un $\mathcal{O}(n^2)$, ma come già detto è molto difficile che venga raggiunto, tanto che il caso medio è proprio un $\Theta(n$
$logn)$. Tuttavia, ci si tiene a dire che l'equazione di ricorrenza che descrive questo algoritmo è:

$$T(n) = T(k) + T(n-k) + \Theta(n)$$

Come vediamo abbiamo un parametro k, che rappresenta l'ampiezza di una singola sequenza per ogni passo. Contrariamente al *Merge Sort*, non possiamo dire che ogni volta si divide in $n/2$, dato che la divisione non è netta ma dipende dal pivot. Per quanto sembri complicata, grazie al metodo di sostituzione possiamo arrivare alla soluzione scritta sopra.

Il vantaggio di questo algoritmo rispetto al *Merge Sort* proposto è chiaramente il fatto che ordina **in loco**, il che è un GRANDE vantaggio, risparmiando parecchio costo spaziale.

Tuttavia non siamo ancora di fronte al miglior algoritmo di ordinamento progettabile. Infatti, per il prossimo algoritmo, dovremo fare una parentesi su una struttura dati che definiremo appositamente per questo algoritmo: stiamo parlando della **struttura dati**

Heap e dell'**algoritmo HeapSort**.

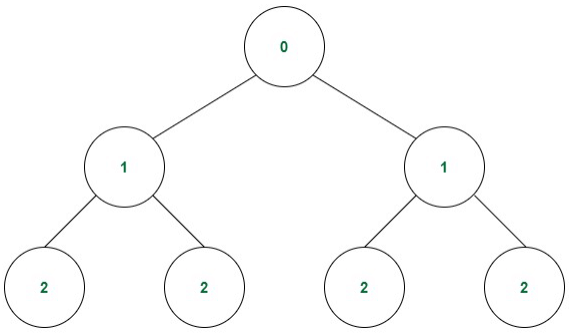
3.2.4 HeapSort

Come tutte le strutture dati (ma lo vedremo meglio in seguito) anche l'Heap è stato pensato per eseguire in modo efficiente alcune poche ma importanti operazioni:

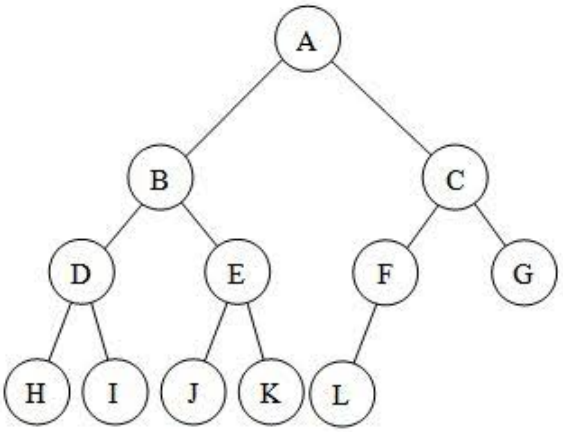
- **Estrazione del massimo**: con estrazione non si intende semplicemente trovarlo e mandarlo a schermo, ma trovarlo e *rimuoverlo* dalla struttura.
- **Inserimento e cancellazione** di un elemento.

Definizione 3.2.1: Albero binario completo e quasi completo

Un **albero binario completo** è un albero che ha il massimo numero di nodi per ogni livello, dunque in cui tutti i livelli sono pieni.
Un **albero binario quasi completo** è un albero che ha il massimo numero di nodi per tutti i livelli ECCETTO CHE per l'ultimo, in cui i nodi sono "concentrati a sinistra".



(a) Albero binario completo



(b) Albero binario *quasi* completo

Possiamo ora passare alla definizione di Heap, di cui però abbiamo due categorie:

Definizione 3.2.2: Heap (Min-Heap e Max-Heap)

Min-Heap: è un albero binario completo o quasi completo in cui viene rispettata la proprietà che la chiave di ogni nodo è minore o uguale alle chiavi dei suoi figli.

Max-Heap: è un albero binario completo o quasi completo in cui viene rispettata

la proprietà che la chiave di ogni nodo è maggiore o uguale alle chiavi dei suoi figli.

L'Heap è una **Struttura Dati Astratta** (SDA, oppure *Abstract Data Structure* in inglese), il che vuol dire che noi ne abbiamo definito solo le proprietà, ma non sappiamo come memorizzarlo e utilizzarlo fisicamente in un programma o un algoritmo.

Difatti, la memorizzazione fisica dell'Heap si fa mediante un array A che possiede le seguenti proprietà:

- Il numero di elementi dell'array A corrisponde al numero di nodi dell'Heap, che chiamiamo infatti `heap_size`.
- La **radice** dell'Heap corrisponde all'elemento $A[0]$.
- Se un nodo si trova in posizione i, possiamo calcolare il suo **figlio sinistro** con la formula $2i + 1$, mentre il **figlio destro** sarà $2i + 2$.
- Per risalire al padre di un nodo, possiamo usare la formula $\lfloor \frac{i-1}{2} \rfloor$, dove le parentesi indicano l'operazione di *floor* (approssimazione per difetto), dato che potremmo ottenere un risultato decimale.

Questo tipo di rappresentazione di un albero in un array, utilizzando gli indici per definire le relazioni di parentela, è detta **"rappresentazione posizionale di un albero"**.

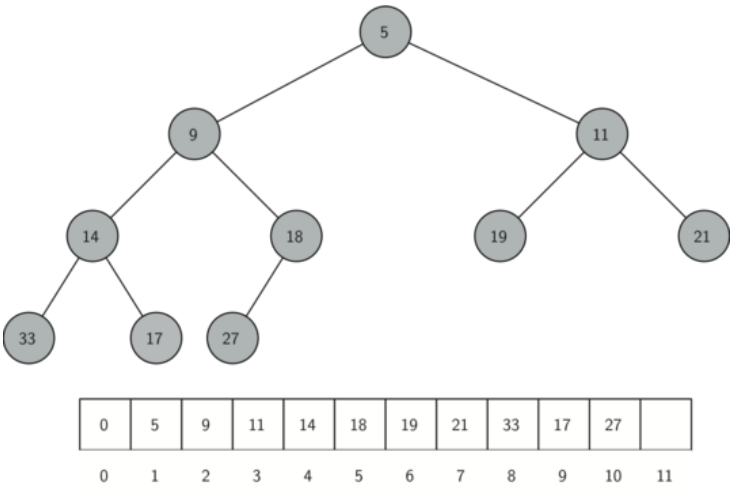


Figura 3.2: Rappresentazione posizionale di un albero binario quasi completo

Come possiamo vedere, scorrere da sinistra a destra l'array significa anche visitare l'Heap per livelli, dall'alto verso il basso. Simmetricamente, scorrere da destra a sinistra significa visitarlo dal basso verso l'alto, scorrendo sempre per livelli.

Facciamo alcune osservazioni sull'Heap:

- Muoversi dalla radice ad una qualsiasi foglia dell'Heap richiederà tempo al massimo pari a $\mathcal{O}($
  $logn$), dove n è il numero di nodi dell'Heap (la `heap_size`) e dunque
  $log(n)$ rappresenta l'altezza dell'albero.
- Se siamo in un Max-Heap, trovare l'elemento massimo è molto facile, è il primo elemento; analogamente, se siamo in un Min-Heap, trovare l'elemento minimo corrisponde a leggere il primo elemento.

Come ogni struttura dati astratta, oltre alle proprietà matematiche che la definiscono, definiamo anche i metodi, cioè le operazioni che la caratterizzano:

Funzione `Heapify`: si tratta di una funzione che si occupa di "ricostruire" l'Heap. Questa ricostruzione avviene sotto l'ipotesi che, sull'albero su cui viene eseguita la funzione, il sottoalbero sinistro e destro della radice siano degli Heap a loro volta. Dunque l'unico nodo che può violare la proprietà di Heap è proprio la radice, che può essere minore di uno dei due figli. A questo punto la funzione scambia la radice col maggiore dei suoi due figli, e poi applica ricorsivamente la funzione `Heapify` considerando come radice il nodo appena scambiato. Il codice della funzione è il seguente:

```
1         def heapify(A, i, heap_size):
2           left = 2*i + 1
3           right = 2*i + 2
4           indice_massimo = i
5
6           if left > heap_size: return
7           if right > heap_size: return
8
9           if A[left] > A[indice_massimo]:
10            indice_massimo = left
11          if A[right] > A[indice_massimo]:
12            indice_massimo = right
13
14          if indice_massimo != i:
15            A[indice_massimo], A[i] = A[i], A[
                 indice_massimo]
16            heapify(A, indice_massimo, heap_size)
```

Listing 3.9: Funzione heapify

Spiegazione algoritmo 3.2.3: Heapify

- Prendiamo in input nella funzione l'array `A` in cui è memorizzato l'Heap, l'indice `i` del nodo corrente e la dimensione dell'heap, `heap_size`.
- Ci calcoliamo, tramite le formule espresse precedentemente, il figlio sinistro e figlio destro del nodo in cui ci troviamo.
- Impostiamo l'indice massimo essere, temporaneamente, il nodo corrente.
- In seguito dobbiamo verificare se uno dei due nodi figli è maggiore del padre. Le due condizioni scritte sono:
  - `left_child/right_child` $<$ `heap_size`: perché non vogliamo prendere valori che sono dentro all'array ma FUORI dall'Heap.
  - `A[left_child]` $>$ `A[indice_massimo]` (oppure per `right_child`): controlliamo se il valore di uno dei due figli supera il valore dell'indice massimo (che inizialmente era il nodo corrente).
- Se entrambe le condizioni sono rispettate, per almeno uno dei due blocchi `if`, il valore di `indice_massimo` viene cambiato.
- Se l'indice contenente il valore massimo non è più quello iniziale (`i`) dobbiamo scambiare il valore all'indice massimo (il figlio) con il valore all'indice corrente (il padre) e poi aggiustare l'Heap ricorsivamente però partendo dal nodo appena scambiato.

Questa funzione opera in tempo logaritmico, in quanto il numero MASSIMO di cambi è pari all'altezza dell'albero. Dunque il costo computazionale sarà $\mathcal{O}(\log(n))$, dove n è la `heap_size`.

Funzione `BuildHeap`: si tratta di una funzione che si occupa di costruire un Heap a partire da un qualsiasi array disordinato. Questa funzione si basa sull'ipotesi che ogni foglia singola sia già un Heap. Dunque la funzione `BuildHeap` non opera dall'alto verso il basso come `Heapify`, bensì dal basso verso l'alto, rendendo le foglie degli Heap prima che lo sia reso il padre. Ricordandoci che nella rappresentazione posizionale dell'Heap è come se scorressimo l'albero per livelli, le foglie sono tutte

all'ultimo livello, e dunque alla destra dell'array. Per capire quante foglie abbiamo ci basta sapere la `heap_size` (n). Siccome in un albero binario completo (o quasi completo) il numero di foglie è circa la metà dei nodi totali dell'albero, abbiamo che il numero di foglie è circa $\lfloor n/2 \rfloor$. Dovendo procedere dal basso verso l'alto (da destra a sinistra se consideriamo l'array), dovremo iniziare a sistemare i nodi a partire dalle foglie e poi salendo. Dovremo dunque scorrere l'array al contrario, partendo dalle foglie, e poi risalendo pian piano.

```
1        def build_heap(A, heap_size):
2          for i in reversed(range(len(A) // 2)):
3            heapify(A, i, heap_size)
```

Listing 3.10: Codice della funzione BuildHeap

Il codice è di conseguenza MOLTO semplice. Con un'analisi poco accurata del codice, si potrebbe dire che il costo computazionale della funzione è un $\mathcal{O}(n \cdot \log(n))$, in quanto la funzione `Heapify` costa $\mathcal{O}(\log(n))$ e viene ripetuta $n/2$ volte. Tuttavia quest'analisi è errata, in quanto `Heapify()` ha costo $\mathcal{O}($
$logn$) solo su alberi di altezza h. In questo caso, però, siccome partiamo dalle foglie, inizialmente `Heapify()` avrà costo 0 (perché una foglia non ha figli), poi avrà costo 2 (perché staremo considerando il sottoalbero che parte dal padre della foglia di prima), poi avrà costo 4, ... e via dicendo.

Dunque il costo all'interno del ciclo è una sommatoria notevole, per un indice h che va da 0 all'altezza massima dell'Heap, cioè
$log(n)$, del numero di nodi che stanno sotto ad un sottoalbero di altezza h. Il numero di nodi in questione è $\frac{n}{2^{h+1}}$. La sommatoria che descrive il costo computazionale della funzione sarà dunque la seguente:

$$\Theta\left(\sum_{h=0}^{logn} \frac{n}{2^{h+1}}\right)$$

Possiamo portare fuori dalla sommatoria $\Theta(n)$, in quanto n non è influenzato dall'indice h, e ci troviamo:

$$\Theta(n) \cdot \Theta\left(\sum_{h=0}^{logn} \frac{1}{2^{h+1}}\right)$$

La seconda sommatoria è una sommatoria notevole, in particolare una serie geometrica generalizzata, che restituisce come valore una costante. Il costo della funzione sarà dunque $\Theta(n)$.

Funzione `Heapify Bottom-Up`: Per inserire un elemento all'interno di un Heap, la cosa più conveniente è inserirlo come ultimo elemento a destra dell'array (quindi un'altra foglia), e poi confrontarlo con il suo padre ricorsivamente per mettere quel valore al posto giusto. Nota che non conviene mettere l'elemento al primo posto, quindi come radice, perché sarebbe da fare uno slittamento di tutti gli elementi dell'array verso destra E POI aggiustare la struttura Heap con `Heapify`. Conviene dunque crearsi una funzione `Heapify_Bottom_Up` che aggiusta la struttura dell'Heap a partire dal basso e salendo verso l'alto. Di seguito il codice della funzione, che verrà poi analizzato:

```
1        def heapify_bottom_up(A, indice_corrente):
2          padre = floor(indice_corrente / 2) - 1
3          if A[padre] < A[indice_corrente]:
4            A[padre], A[indice_corrente] = A[
                indice_corrente], A[padre]
5            if padre > 0:
6              heapify_bottom_up(A, padre)
```

Listing 3.11: Codice della funzione Heapify Bottom-Up

Il codice lavora nel seguente modo:

Spiegazione algoritmo 3.2.4: Heapify Bottom-Up

- Alla primissima chiamata, come parametri della funzione noi avremo `A`, la rappresentazione fisica dell'Heap tramite array, e `heap_size` -1, perché appunto il primissimo elemento che dobbiamo considerare sarà l'ultimo dell'array, che corrisponde all'indice `heap_size` -1.
- Ci calcoliamo il padre dell'indice corrente con la formula che già conosciamo.
- Se il padre è minore dell'indice corrente (RICORDA!!! Stiamo partendo dal basso, dunque l'indice corrente è quello del figlio) allora li scambiamo.
- Procediamo con la ricorsione solo se l'indice padre è maggiore di 0 (questo perché se fosse uguale a 0 non avrebbe senso continuare dato che saremmo arrivati alla radice).

Come si può intuire, il costo computazionale di questa funzione è pari a quello del normale `Heapify()`, cioè un $\mathcal{O}(\log(n))$, in quanto il numero massimo di scambi che vengono fatti è pari all'altezza dell'Heap.

Avendo fatto questa GRANDE parentesi sulla struttura dati Heap, possiamo procedere con il motivo che ci ha spinto a definirla: l'algoritmo di ordinamento *Heap Sort*.

L'algoritmo *Heap Sort* è concettualmente molto semplice, e ricorda per certi versi il *Selection Sort*: cerchiamo il valore massimo dell'Heap e lo scambiamo con l'ultimo valore. Poi ricostruiamo l'Heap con `Heapify`, in modo che mantenga le proprietà di un Heap e l'algoritmo possa continuare a funzionare correttamente. Anche stavolta il codice è molto semplice, e si sviscera in poche righe:

```
1       def heap_sort(A):
2         n = len(A)
3         build_heap(A, n)
4
5         for i in reversed(range(1, n)):
6           A[0], A[i] = A[i], A[0]
7           heapify(A, 0, i)
```

Listing 3.12: Codice dell'algoritmo Heap Sort

Analizziamo il codice:

Spiegazione algoritmo 3.2.5: HeapSort

- Come primissima cosa, vogliamo trasformare il nostro array di partenza in un Heap: altrimenti non avrebbe avuto senso fare tutti i discorsi fatti prima.
- Poi cominciamo ad iterare dall'ULTIMO ELEMENTO FINO AL PRIMO, cioè da DESTRA VERSO SINISTRA. Perché facciamo questo? Perché abbiamo detto che progressivamente vogliamo scambiare la radice con l'ultimo elemento dell'Heap.
- Nota che, se consideriamo `len(A)` $= n$, il ciclo prenderà tutti i valori che vanno da $n, n-1, n-2, \ldots, 1$. Dunque procediamo allo scambio. Senza guardare per ora la chiamata di `Heapify()`, alla prima iterazione viene scambiato il primo elemento con l'elemento ad indice $n-1$; alla seconda viene scambiato il primo con $n-2$; alla terza il primo con $n-3$; e così via. Scambiamo il primo elemento proprio perché sappiamo che in un Max-Heap il primo elemento è il

massimo, quindi se lo mettiamo in fondo stiamo ordinando l'array a partire da destra verso sinistra.

- Ora arriva la parte più "ostica": la chiamata `Heapify()`. Ricordiamo che i parametri di `Heapify()` sono, in ordine: l'array che rappresenta l'Heap, l'indice da cui dobbiamo iniziare a ricostruire l'Heap, e la `heap_size`. Come vediamo, in questo caso l'indice da cui dobbiamo partire a ricostruire l'Heap è sempre l'indice 0, perché è l'indice in cui mettiamo la foglia con cui abbiamo scambiato la radice, ed è dunque l'indice che viola la proprietà di Heap. Poi come terzo parametro mettiamo `i`. Ricordiamo che `i` vale $n-1, n-2, n-3, \ldots$ Usiamo proprio questo valore, perché progressivamente vogliamo DIMINUIRE la dimensione dell'Heap, fino a renderlo di un solo elemento. Perché se lasciassimo la dimensione sempre costante, il nostro "ordinamento" restituirebbe l'Heap iniziale. Se invece ogni volta decrementiamo la dimensione dell'Heap da ricostruire, è come se gli stessimo dicendo: *"Ricostruisci l'Heap, ma fino agli elementi che hai già ordinato. Quelli non toccarli, perché sono ordinati e stanno bene così"*. Capito questo concetto, l'Heap Sort è concluso.

Come già detto quando è stato introdotto questo argomento, questo algoritmo è la conclusione di un percorso relativo a degli algoritmi di ordinamento: opera in tempo $\Theta(n \cdot \log(n))$ e ordina **in loco**: tutto ciò che abbiamo sempre voluto.

Di seguito si lascia una tabella di confronto dei maggiori algoritmi di ordinamento, con costo computazionale nel caso migliore e caso peggiore, casi d'uso, vantaggi e svantaggi.

Algoritmo	**Costo temporale**	**Costo spaziale**	**Stabile?**
Merge Sort	Best Case, Worst Case: $\Theta(n \cdot \log(n))$	$O(n)$	Si
Quick Sort	Best Case: $\mathcal{O}(n \cdot \log(n))$, Worst Case: $\Theta(n^2)$	$\mathcal{O}(\log(n))$	No
Heap Sort	Best Case, Worst Case: $\Theta(n \cdot \log(n))$	$\mathcal{O}(\log(n))$	No

3.3 Ordinamenti lineari

Il titolo è volutamente provocatorio: cosa vuol dire che si può ordinare in tempo lineare? Non avevamo dimostrato, grazie all'albero decisionale, che il limite inferiore per gli algoritmi di ordinamento è di $\Omega(n$
$logn)$? E allora come è possibile che ci siano algoritmi che ordinano in tempo lineare?

Beh, è presto detto, in quanto bisogna ricordarsi che il limite inferiore vale **solo per gli algoritmi di ordinamento basati sui confronti**. Dunque non concerne alcun altro tipo di algoritmo che, ad esempio, pone delle strette ipotesi sulla natura dei dati. Di seguito proponiamo due algoritmi che rispettano questa definizione.

3.3.1 Counting Sort

Si basa sull'ipotesi che, in un array A, siano contenuti i **valori compresi in un intervallo del tipo** $[\mathbf{0}, \mathbf{k}]$, oppure $[1, k]$, $[-k, k]$, ... (si può leggermente variare, ma il concetto di base è quello). Il costo computazionale di questo algoritmo sarà un $\Theta(n + k)$ e, se $k = \mathcal{O}(n)$, allora possiamo dire che il costo è lineare $\Theta(n)$.

L'idea dietro a questo algoritmo è che ogni elemento appartenente all'intervallo determini univocamente la sua posizione nell'array ordinato, **senza** fare confronti, ovviamente. Dovremo creare un array di appoggio, che chiamiamo C, che avrà dimensione $k + 1$ (cioè il numero di elementi presenti in un generico intervallo del tipo $[0, k]$). In ogni posizione di questo array C verranno messe le occorrenze di ogni singolo elemento che dovrebbe stare in quella posizione.

Ad esempio, se abbiamo un array $A = [0, 6, 7, 2, 5, 6, 1, 0, 4, 4, 1, 6]$ notiamo come ci siano dei duplicati. Se prendiamo il massimo elemento di A, vediamo che questo è 7. Dunque dovremo andare ad inizializzare un array C di $7 + 1 = 8$ elementi. Nella posizione 0 dell'array C metteremo il numero di occorrenze del primo elemento dell'intervallo: siccome sappiamo che l'intervallo parte da 0, nella posizione 0 dovremmo metterci quante volte appare l'elemento 0 nell'array A. Il concetto è analogo per tutti gli altri elementi. Alla fine della fiera, l'array C sarà $C = [2, 2, 1, 0, 2, 1, 3, 1]$. Dovremo poi vedere come mettere questi elementi all'interno dell'array A.

Vediamo dunque il codice relativo all'algoritmo:

```
1       def counting_sort(A):
2         k = max(A)
3         n = len(A)
4         C = [0] * (k+1)
```

```
5
6           for j in range(n):
7             C[A[j]] += 1
8
9           j = 0
10          for i in range(k):
11            while C[j] > 0:
12              A[j] = i
13              j += 1
14              C[i] -= 1
```

Listing 3.13: Codice base dell'algoritmo Counting Sort

Analizziamolo nel dettaglio:

Spiegazione algoritmo 3.3.1: Counting Sort

- Come prima cosa otteniamo il massimo di A, per capire fino a dove si estende l'intervallo che stiamo considerando.
- Creiamo l'array C, di lunghezza $k+1$, cioè il numero di elementi dell'intervallo, e per ora lo inizializziamo a tutti 0.
- Cicliamo per tutti gli elementi dell'array A. Per ogni elemento dell'array A aumentiamo di uno la sua corrispettiva posizione all'interno dell'array C. Guardare l'esempio precedente per ulteriori chiarimenti.

Adesso dobbiamo sistemare l'array originale, facendo in modo che risulti ordinato:

- Cicliamo per ogni elemento all'interno dell'array C.
- Siccome ogni elemento di C contiene **quanti elementi di quel numero sono presenti in** A, dobbiamo ciclare di nuovo finché il conteggio nel singolo elemento di C non si esaurisce. Dunque facciamo partire un ciclo che continua finché il singolo elemento non ha un numero di occorrenze pari a 0 (perché verranno progressivamente decrementate).
- A questo punto, possiamo modificare l'array originale A mettendo all'indice j (che **non è** l'indice del ciclo, bensì un indice che viene incrementato solo quando bisogna effettivamente mettere qualcosa dentro A) l'indice dell'elemento corrente presente in C.

- Ora decrementiamo il conto di quell'elemento all'indice i nell'array C, in modo da sapere se era presente un'altra occorrenza di quell'elemento oppure se possiamo procedere.
- Incrementiamo di 1 l'indice j in modo da permettere di salvare un elemento in posizione successiva nell'array A.

Nota che questo algoritmo ha un costo lineare se e solo se $k = \mathcal{O}(n)$, e inoltre un suo grande difetto (oltre all'importantissima supposizione sulla natura dei dati) è che, così come l'abbiamo scritto, **non salva** i dati satellite correlati ad un elemento: se avessimo dei record ognuno dei quali ha dei dati, questi dati andrebbero persi oppure assegnati ad un altro elemento.

Di seguito si propone una soluzione di *Counting Sort* con la gestione dei dati satellite: servirà un ulteriore array di appoggio per permettere di conservare i dati. Notiamo dunque che, se stiamo risparmiando in costo temporale, ne stiamo consumando parecchio in costo spaziale.

```
1   def counting_sort_dati_satellite(A):
2     k = max(A)
3     n = len(A)
4     C = [0] * (k+1)
5     B = [0] * n
6
7     for j in range(n):
8       C[A[j]] += 1
9     for i in range(1, k+1):
10      C[i] += C[i-1]
11    for j in reversed(range(n)):
12      B[C[A[j]]-1] = A[j]
13      C[A[j]] -= 1
14
15    return B
```

Listing 3.14: Codice del Counting Sort con salvataggio dei dati satellite

In questa implementazione, le prime righe rimangono le stesse, solo che l'array C, oltre a contenere il numero di occorrenze di un singolo elemento dell'intervallo nella corrispettiva posizione i, contiene anche (per il secondo ciclo `for`) per ogni elemento, il numero di elementi che stanno prima di lui.

Tornando sempre all'esempio di $A = [0, 6, 7, 2, 5, 6, 1, 0, 4, 4, 1, 6]$, dopo il primo ciclo `for`, C sarà $[2, 2, 1, 0, 2, 1, 3, 1]$. Dopo il **secondo** ciclo, avremo che ogni elemento di C rappresenterà quanti elementi ci sono prima di lui. C diventerà quindi $C = [2, 4, 5, 5, 7, 8, 11, 12]$. Quest'intuizione è geniale, perché ci permette di capire in che posizione dovranno finire i valori senza doverli modificare.

Ad esempio, sapendo che in $C[2]$ abbiamo 4, significa che il numero 2 avrà 3 elementi prima di lui, con lui che è appunto il quarto, e dunque l'ultimo 2 andrà in posizione $C[2] - 1 = 3$. Vale lo stesso per il numero 4, ad esempio. Questo dovrebbe essere inizialmente ad indice 4, ovviamente, ma siccome prima di lui ci sono 7 elementi, verrà messo in posizione $C[4] - 1 = 6$. Ci riferiamo ovviamente all'ultimo 4 che appare. Poi per trovare quelli prima è semplice: sappiamo quanti sono e andiamo a ritroso.

Il ciclo finale, eseguito al contrario, serve proprio per fare questo: partendo dall'ultimo elemento, vediamo in che posizione metterlo in base a quanti elementi ci sono prima di lui. Sappiamo già che numero rappresenta grazie all'ipotesi fatta, ci resta solo da capire dove metterlo in posizione corretta. Stiamo posizionando i dati nell'array B proprio perché non vogliamo modificare in alcun modo A, per non perderne i dati.

3.3.2 Bucket Sort

Un altro algoritmo di ordinamento che viene eseguito in tempo lineare e fa ipotesi sulla natura dell'input è il *Bucket Sort*. In particolare, assume che gli n elementi da ordinare siano **equamente distribuiti in un intervallo del tipo** $[\mathbf{1}, \mathbf{k}]$, senza alcuna ipotesi su k.

Fatta questa ipotesi, l'idea è quella di dividere l'intervallo $[1, k]$ in n sottointervalli, ciascuno di ampiezza k/n, detti *bucket*, per l'appunto.

Ad esempio, se partiamo con un array $A = [8, 6, 15, 2, 5, 16, 4, 9]$, abbiamo che l'intervallo sarà $[1, 16]$ e dunque dovremo dividerlo in 8 sottointervalli, ognuno di ampiezza $16/8 = 2$. L'array B sarà dunque sempre di 8 elementi come l'array A originale, ma al suo interno conterrà delle **liste** di elementi (appunto i bucket) all'interno delle quali inseriremo gli elementi di A (chiaramente se presenti).

Non viene presentato il codice in quanto richiederebbe l'uso di strutture dati più complesse non ancora trattate.

4

Strutture dati

Passiamo ora al secondo macro-argomento di questo corso: le **strutture dati**. Nonostante ne abbiamo, ovviamente, già incontrate alcune nel nostro percorso riguardante gli algoritmi di ricerca (come l'array, banalmente, o anche l'Heap), si ritiene necessario fornire una definizione formale di struttura dati e le loro classificazioni.

Innanzitutto, prima di definire cos'è una struttura dati, definiamo che cos'è un dato.

Definizione 4.0.1: Dato

In un linguaggio di programmazione, un dato è banalmente il valore che può essere assunto da una variabile. Tuttavia, prima di arrivare alla "pratica" dei linguaggi di programmazione, bisogna passare dalla teoria dei dati.
Infatti, prima di aver definito i tipi di dati che conosciamo nei linguaggi di programmazione (`int`, `float`, `double`, `String`, ...) viene definito, concettualmente, il **tipo di dato astratto** (*Abstract Data Type*). Il tipo di dato astratto è un modello matematico, definito da un dominio di valori che possono essere assunti da quel tipo di dato, e dalle operazioni definite su quel tipo di dato.
Un esempio concreto potrebbe essere il tipo di dato dei **numeri interi**: il dominio di questo tipo di dato è chiaramente l'insieme numerico $\mathbb{Z}$ dei numeri interi (concettualmente è così, nella pratica è impossibile rappresentare infiniti numeri in un calcolatore); l'insieme delle operazioni definite per questo tipo di dato sono ad esempio la somma, la moltiplicazione, la sottrazione, l'elevamento a potenza, la divisione, ...

Ora che abbiamo definito formalmente cosa è un dato, almeno nella teoria, vediamo cosa si intende per "struttura dati".

Definizione 4.0.2: Struttura Dati

Una struttura dati è un particolare tipo di dato (astratto, ovviamente, non esistono fisicamente gli array nel mondo, per intenderci) caratterizzato dal fatto che "organizza" (oppure si potrebbe dire "contiene") altri dati al suo interno.
Essendo una struttura dati un tipo di dato a sua volta, anche questa avrà:

- Un modo sistematico di organizzare i dati (sarebbe la sua versione del "dominio" di dati assumibile);
- Un insieme di operazioni che permettono di interagire con essa.

Possiamo fornire una classificazione delle strutture dati in base ad alcune loro caratteristiche:

Lineari o non lineari: strutture dati in cui possiamo o meno definire una sequenzializzazione (cioè possiamo definire un primo, secondo, terzo, ... elemento). Ad esempio, l'array indicizzato è una struttura lineare, mentre l'albero ad esempio è non lineare.

Statiche o dinamiche: sono statiche le strutture in cui abbiamo una dimensione costante, che non può variare nel tempo. Viceversa le dinamiche, se nel tempo possiamo aggiungere elementi senza troppi problemi.

Omogenee o disomogenee: omogenee nel caso in cui tutti gli elementi appartengano allo stesso tipo di dato, oppure disomogenee se ci potrebbero essere elementi appartenenti a tipi di dato diversi.

Una struttura dati serve a memorizzare e manipolare **insiemi dinamici**, cioè insiemi i cui elementi possono variare nel tempo, a seconda delle operazioni che vengono compiute sulla struttura. Gli elementi dell'insieme dinamico, spesso chiamati *nodi*, *record* o *oggetti*, possono essere a loro volta molto complessi, e contenere anche loro diversi tipi di dati "elementari". In tal caso possiamo distinguere due componenti principali di questi elementi:

- Una **chiave**, cioè un identificativo **univoco** relativo a ciascun elemento dell'insieme dinamico. È un campo indispensabile per permettere di compiere operazioni relative ad un singolo elemento (ad esempio, parlando degli array, la chiave potrebbe essere l'indice dell'elemento, dato che non possono esserci più di un elemento a quell'indice).
- Ulteriori dati, detti **dati satellite**, relativi all'elemento stesso, che non è detto siano univoci.

Su un insieme dinamico possiamo definire due categorie di operazioni che possono essere compiute: operazioni di **interrogazione** (cioè operazioni che non modificano l'insieme dinamico e i suoi elementi, ma servono solo per ottenere informazioni su essi) e operazioni di **modifica** (che, come suggerisce il nome, servono a modificare un elemento dell'insieme dinamico).

Tipici esempi di operazioni di interrogazione sono:

- Ricerca di un dato elemento k all'interno dell'insieme $S \rightarrow \texttt{Search}(S, k)$;
- Ricerca del valore minimo nell'insieme S (chiaramente il significato di "minimo" dipende dalle relazioni di ordine stabilite sull'insieme che stiamo considerando: il minimo di interi non sarà il minimo di due stringhe, ad esempio) $\rightarrow \texttt{Min}(S)$;
- Ricerca del valore massimo dell'insieme S (stesso discorso di prima) $\rightarrow \texttt{Max}(S)$;
- Recupero dell'elemento che precederebbe un elemento dato k, qualora l'insieme fosse ordinato $\rightarrow \texttt{Pred}(S, k)$;
- Recupero dell'elemento che seguirebbe un elemento dato k, qualora l'insieme fosse ordinato $\rightarrow \texttt{Succ}(S, k)$.

Esempi di operazioni di modifica sono invece:

- Inserimento di un valore k nell'insieme $S \rightarrow \texttt{Insert}(S, k)$;
- Eliminazione di un elemento k del quale supponiamo di conoscere la locazione $\rightarrow$ $\texttt{Delete}(S, k)$.

Possiamo fare un piccolo appunto sulle strutture dati che incontreremo d'ora in poi: sono tutte strutture dati astratte (*Abstract Data Structures*), cioè sono oggetti matematici di cui noi definiamo le proprietà e le operazioni, ma che nella pratica spesso e volentieri non possono essere implementate "direttamente".

Strutture dati come l'Array e le Liste concatenate possiamo definirle come i "mattoni" per l'implementazione fisica delle altre strutture, dato che sono le uniche due strutture che hanno un modo di essere memorizzato in memoria molto naturale: gli Array sono sequenze di valori "uno dopo l'altro", mentre le Liste concatenate collegano i vari elementi tra loro mediante dei puntatori ad altre locazioni di memoria. Altre strutture più avanzate, come la Coda, la Pila, gli Alberi o i Dizionari non sono altro che strutture che non esistono nella realtà, con delle proprietà specifiche, sì, ma che nel pratico vengono implementate con Array o Liste concatenate gestite in un modo particolare.

4.1 Array

L'array è forse la struttura dati più semplice che si potrebbe pensare, e non a caso è anche la base di tutte le strutture dati che verranno dopo, e di quasi tutte le loro implementazioni fisiche. Partiamo col definire l'array come:

- Una **sequenza ordinata** (è dunque una struttura lineare, in cui ogni elemento ha un indice che ne identifica la posizione e dunque è stabilito un ordine) di elementi omogenei;
- Una struttura dati **statica**;
- Una struttura dati ad **accesso casuale**: conoscendo l'indice di un elemento, possiamo accedervi in tempo costante, indipendentemente da dove questo indice sia.

Nonostante l'Array sia una struttura molto semplice, ne distinguiamo comunque due categorie che hanno importanti differenze tra loro: **Array semplice** (o disordinato) e **Array ordinato**. Queste due differiscono nel costo computazionale di alcune delle operazioni che possono essere effettuate su di essi. Vediamoli a confronto in una tabella:

Struttura dati	**Ricerca**	**Min**	**Max**	**Insert**	**Delete**
Array disordinato	$O(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Array ordinato	$\Theta(\log(n))$	$\Theta(1)$	$\Theta(1)$	$O(n)$	$O(n)$

Come si può vedere, anche tra due strutture dati che all'apparenza sono la stessa sono presenti delle importanti differenze. Andiamo a spiegare il perché di alcune operazioni dal costo particolare:

Minimo e massimo: in un array disordinato, dovremo necessariamente scorrere tutti gli elementi per trovare il minimo e il massimo, mentre in un array già ordinato (supponiamo in ordine crescente) questi saranno rispettivamente al primo e all'ultimo indice.

Inserimento: in un array qualsiasi non è importante mantenere alcun ordine, e dunque possiamo aggiungere un nuovo elemento in coda all'array senza troppe storie. In un array ordinato, invece, dobbiamo posizionare l'elemento all'indice corretto, dunque dovremo scorrere l'array da sinistra a destra per cercare il posto giusto al nostro elemento, facendo poi "slittare" tutti gli altri elementi di un posto a destra.

Cancellazione: in un array qualsiasi possiamo scambiare l'elemento all'indice da eliminare con l'ultimo, per poi eliminare l'ultimo e non dover neanche effettuare uno slittamento verso sinistra. In un array ordinato invece questo deve accadere, in quanto deve appunto essere preservato l'ordine.

4.2 Lista concatenata semplice

Prima di parlare di lista concatenata semplice, facciamo una parentesi sul concetto di "puntatore". In molti linguaggi di programmazione più verso il basso livello esistono questi particolari tipi di dato (chiamati per l'appunto "puntatori") che hanno come scopo quello di memorizzare (oppure proprio "puntare", da qui il nome) l'indirizzo di memoria di un'altra variabile in utilizzo nel programma. Sostanzialmente sono tipi di dato che memorizzano indirizzi di memoria al loro interno, e grazie a loro possiamo raggiungere anche manualmente un dato memorizzato ad un dato indirizzo.

Questo concetto è di fondamentale importanza se vogliamo capire le **liste concatenate semplici** (in inglese *Single Linked List*). Si tratta di una struttura dati in cui ogni elemento contiene due campi principali:

- La sua chiave (`key`) e (occasionalmente) i dati satellite ad essa associati, che identificano univocamente l'elemento;
- Un puntatore al prossimo elemento della lista concatenata (`next`). In caso dell'ultimo elemento della lista, questo avrà come puntatore `Null` (o `None`, dipende dall'implementazione che vorremmo dare).

Inoltre, una lista puntata ha due importantissime proprietà principali:

- L'accesso avviene **sempre e solo ad un'estremità** della lista. Infatti, ogni volta che si lavora con le liste puntate si passa come parametro il puntatore all'elemento che sta a testa della lista.
- L'accesso è permesso **SOLO in maniera sequenziale**. Qui la principale differenza con gli Array: se negli array per accedere ad un elemento ad indice i possiamo farlo in tempo costante, qui dobbiamo scorrere tutti gli elementi FINO ad arrivare all'elemento a cui volevamo accedere.

Nonostante queste due differenze possano sembrare molto svantaggiose, il principale vantaggio delle liste puntate è che sono strutture **dinamiche**: per aggiungere un elemento, basta modificare il campo `next` per aggiungere un riferimento ad un nuovo elemento.

Difatti, per aggiungere un elemento, si può intuire che non conviene aggiungerlo alla fine della lista in quanto, non essendo consentito l'accesso diretto, si dovrebbero scorrere tutti gli elementi della lista. Dunque, per aggiungere un elemento ad una lista puntata conviene farlo alla testa della lista, facendo in modo che il campo `next` del nuovo elemento punti a quella che prima era la testa della lista: basta. Il codice è il seguente:

```
1       def inserimento(p, k):
2         if k != None:
3           k.next = p
4           p = k
5         return p
```

Listing 4.1: Codice per l'inserimento in testa in una lista concatenata semplice

In questo caso, `p` è il puntatore alla testa della lista (quello da cui facciamo l'accesso sempre) e `k` è il nuovo record che vogliamo inserire. Il processo è molto semplice:

Spiegazione algoritmo 4.2.1: Inserimento in lista semplice

- Facciamo in modo che il campo `next` del nuovo record punti alla testa della lista, in modo che così la testa della lista sia ora puntata e raggiungibile.
- Siccome vogliamo mantenere, per notazione, che `p` è il puntatore alla testa della lista, assegniamo che `p = k` proprio per fare in modo che il puntatore alla testa si chiami sempre `p`, ma abbia i valori dei campi contenuti in `k`.

Si può semplicemente osservare come il costo computazionale di un inserimento in testa sia di $\Theta(1)$. Possiamo invece immaginare che sarebbe stato un $\Theta(n)$ se avessimo voluto inserire l'elemento in coda, in quanto avremmo dovuto scorrere tutti gli elementi E POI fare `ultimo.next = k`. Chiaramente non ne valeva la pena.

Tuttavia, proprio a causa dell'impossibilità di un accesso diretto agli elementi, la ricerca richiederà intuitivamente un tempo pari a $\mathcal{O}(n)$, in quanto nel peggiore dei casi per trovare un elemento dovremo scorrere tutti gli elementi della lista puntata. Il codice è tuttavia molto semplice, e viene lasciato di seguito:

```
1       def ricerca(p, k):
2         if p is None: return -1
3
4         p_corr = p
```

```
5         while p_corr.key != k and p_corr != None:
6           p_corr = p_corr.next
7         return p_corr
```

Listing 4.2: Ricerca in una lista concatenata semplice

L'unica cosa che può risultare ambigua in questo codice è l'utilizzo di un puntatore di appoggio `p_corr`. Questo viene fatto in quanto, se assegnassimo `p = p.next` avremmo che la testa della lista si sposterebbe avanti, e anzi rischierebbe di diventare addirittura `None`, se arrivassimo fino alla fine del ciclo. In tal caso perderemmo il nostro unico modo di accedere alla lista concatenata (cosa che ovviamente NON vogliamo che accada). Utilizziamo dunque un puntatore ausiliario, e sarà quello ad incrementarsi man mano che procediamo.

Proponiamo adesso il codice per cancellare un elemento `k` dalla lista (supponiamo di sapere che è presente), che anche stavolta avrà un costo pari ad $\mathcal{O}(n)$, in quanto non possiamo accedere direttamente all'elemento ed eliminarlo. Per rendere la comprensione del codice più intuitiva, supponiamo di avere una lista fatta in questo modo:

$$\texttt{p} \to \texttt{a} \to \cdots \to \texttt{b} \to \texttt{k} \to \texttt{c} \to \cdots \to \texttt{end}$$

```
1       def cancellazione(p, k):
2         if p is None: return p
3
4         if p == k:
5           return p.next
6
7         p_corr = p
8         while p_corr.next.key != k.key:
9           p_corr = p_corr.next
10
11        p_corr.next = k.next
```

Listing 4.3: Cancellazione da una lista concatenata semplice

Spiegazione algoritmo 4.2.2: Cancellazione da lista semplice

- Il primo controllo viene fatto per il caso in cui stessimo cercando di eliminare il primo elemento della lista, la testa. In quel caso dovremo solo spostare

il puntatore della testa dal primo elemento a quello dopo. Ecco perché `p = p.next`, perché così `p` non è più reperibile in alcun modo dato che ne abbiamo cancellato il riferimento.

- Anche qui utilizziamo un puntatore ausiliario `p_corr` come prima.
- Stavolta, tuttavia, non dovremo iterare utilizzando `p_corr` stesso, bensì il suo `next`. Questo perché, volendo eliminare un elemento, non possiamo arrivare all'elemento stesso, altrimenti non avremo modo di stabilire un legame con la parte precedente della lista. Con l'ipotesi che l'elemento con chiave `k` sia effettivamente presente nella lista, ci fermiamo dunque quando arriviamo all'elemento immediatamente precedente a `k`. Nel caso del nostro esempio, ci stiamo fermando a `b`.
- A questo punto la rimozione è molto semplice: facciamo in modo che il campo `next` di `b` non punti più a `k`, ma a quello che sta dopo `k`.

La rimozione di un elemento qualsiasi da una lista concatenata non deve essere vista come una "cancellazione dalla memoria", bensì come un modo di **RECIDERE IL LEGAME** tra quell'elemento e la lista stessa. E recidere il legame significa proprio fare in modo che NESSUN ELEMENTO punti più all'elemento che vogliamo eliminare.

Un'osservazione ora sull'oggetto "list" del Python: questo è un miscuglio tra gli Array (in quanto è possibile effettuare l'accesso diretto agli elementi, mediante indici) e le Liste concatenate, in quanto sono strutture dinamiche e in memoria vengono memorizzate tramite puntatori. I metodi che Python mette a disposizione sono i seguenti, con relativi costi:

- `append()`: aggiunge in coda, $\Theta(1)$;
- `insert(i, k)`: inserisce elemento `k` in posizione `i`, costo $\mathcal{O}(n)$ in quanto come negli Array deve fare slittare tutti gli elementi alla destra di `i`;
- `pop(i)`: elimina elemento in posizione `i`, dunque anche qui $\mathcal{O}(n)$ in quanto dovrà essere fatto lo slittamento.

4.3 Lista concatenata doppia

Si tratta di una variazione delle Liste concatenate semplici, in cui ogni record non contiene solo il puntatore all'elemento successivo ma anche al precedente. Questo campo, intuitivamente, si chiamerà `prev`.

Questo aggiunge un forte vantaggio alle liste concatenate doppie: lo scorrimento potrà essere fatto sia da sinistra a destra CHE da destra a sinistra, contrariamente a prima dove ci si poteva muovere solo da sinistra a destra. Le operazioni di ricerca e di inserimento sono analoghe a quelle delle Liste concatenate semplici (con dovuti accorgimenti nella gestione del nuovo campo `prev`), ma ciò che cambia davvero è l'operazione di cancellazione, che stavolta viene eseguita in tempo **COSTANTE** $\Theta(1)$.

```
def cancellazione(p, k):
  if k.prev != None:
    k.prev.next = k.next
  else:
    p = k.next
  if k.next != None:
    k.next.prev = k.prev
  return p
```

Listing 4.4: Cancellazione in una lista concatenata doppia

Abbiamo che `k.prev = None` solo se `k = p`. Dunque quando `k = p`, dobbiamo solo spostare la testa della lista in avanti, come facevamo prima. Se invece `k` $\neq$ `p`, abbiamo che `k.prev` $\neq$ `None`, e dunque facciamo in modo che l'elemento PRIMA di `k` punti all'elemento DOPO `k`. In questo modo abbiamo isolato `k` e ora non ha più legame tramite il puntatore `next`.

Gestiamo ora il caso per il puntatore all'elemento precedente. Se `k.next` $\neq$ `None` significa che `k` non è l'ultimo elemento (perché l'ultimo elemento ha campo `next = None`). A quel punto dobbiamo fare in modo che il puntatore `prev` dell'elemento DOPO `k` punti all'elemento PRIMA di `k`. In questo modo abbiamo reciso il legame anche coi puntatori verso sinistra.

Avendo finito il discorso sulle Liste concatenate, si lascia di seguito una tabella con i vari costi computazionali delle operazioni, a confronto fra Liste concatenate semplici e doppie:

Struttura dati	**Ricerca**	**Min/Max**	**Pred/Succ**	**Insert**	**Delete**
Lista semplice	$O(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$O(n)$
Lista doppia	$O(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$

4.4 Coda (Queue)

La coda è una struttura dati astratta che opera secondo un particolare comportamento di *First In First Out* (FIFO): ciò significa che il primo elemento ad entrare nella coda sarà anche il primo elemento ad uscire. Se cerchiamo un'analogia con il mondo reale, possiamo pensare alle code di persone in attesa ad uno sportello, dove la prima che ha preso il numeretto sarà anche la prima ad essere "servita". Se volessimo invece proporre un esempio sempre nel campo dell'informatica, possiamo pensare alla "coda di stampa", che gestisce l'ordine in cui dei documenti devono essere mandati in stampa: il primo ad essere entrato è il primo che verrà stampato.

Su una coda sono definite solo due operazioni:

- Inserimento (o *Enqueue*);
- Estrazione (o *Dequeue*).

La particolarità della coda è che questa viene individuata da due puntatori posti ad estremità opposte:

- `head`, che punta alla testa della coda, ed è anche dove vengono rimossi i dati (dove applichiamo la *Dequeue*);
- `tail`, che punta alla "fine" della coda, ed è anche da dove vengono inseriti i dati (dove applichiamo l'*Enqueue*).

Si è messo volutamente un appunto sulla parola "fine", in quanto la coda si tratta di una struttura dati che vorrebbe essere dinamica. Ovviamente, nel pratico, questo dipende dall'implementazione che le si vuole dare, se tramite Array o tramite Liste puntate.

Visivamente, una coda può essere rappresentata in questo modo:

$$\texttt{head} \to \texttt{a} \to \texttt{b} \to \cdots \to \texttt{tail} \to \texttt{None}$$

Ragioniamo su come potrebbe essere un'implementazione delle code mediante Liste concatenate: sappiamo che su una lista concatenata, l'operazione di inserimento è molto semplice e costa tempo costante, mentre la rimozione di un elemento (che per la coda viene fatta alla fine della struttura) costerebbe $\Theta(n)$. Ricordiamoci dunque che una coda è definita però da due puntatori: `head` e `tail`. Sappiamo che l'operazione di *Enqueue* avviene sulla coda e quella di *Dequeue* sulla testa.

```
def enqueue(head, tail, e):
  if tail == None:
    head = e
    tail = e
  else:
    tail.next = e
    tail = e
  return head, tail
```

Listing 4.5: Operazione di Enqueue (Inserimento) in una coda

Come vediamo, l'*Enqueue* è molto semplice: se la `tail` è effettivamente l'ultimo elemento e ne abbiamo il puntatore (**Importantissimo!** Nelle Liste concatenate semplici non avevamo il puntatore all'ultimo elemento ma solo al primo, dunque l'inserimento alla fine sarebbe stato un grandissimo spreco di risorse, mentre qui possiamo permettercelo benissimo) l'inserimento consiste semplicemente nell'aggiungere l'elemento come campo `next` della `tail` attuale, e poi cambiare la `tail` ad essere `e`, in quanto è quello a stare alla fine adesso.

```
def dequeue(head, tail):
  if head == None:
    return None, None, None

  e = head
  head = head.next
  if head == None:
    tail = None
  return head, tail, e
```

Listing 4.6: Operazione di Dequeue (Estrazione) da una coda

L'operazione di *Dequeue* è altrettanto semplice, e funziona come quando rimuovevamo un elemento dalla testa di una Lista concatenata.

Si noti come il costo computazionale di entrambe le operazioni sia costante, cioè $\Theta(1)$.

Sopra abbiamo implementato la coda mediante liste puntate in quanto risulta più intuitivo, avendo due puntatori e operazioni di simile costo e procedura. Per implementare invece una coda mediante Array, potremmo pensare che sarebbe di natura più svantaggioso: gli Array disordinati permettono inserimento ed estrazione in tempo costante, se operiamo

alla fine dell'Array, ma la coda richiede che questo sia fatto da due estremità opposte. Se scegliamo dunque di avere un inserimento veloce, dovremo avere un'estrazione lenta (in quanto dovremmo rimuovere l'elemento ad indice 0 e poi "shiftare" tutti gli elementi verso sinistra, ricoprendo l'indice rimosso); il discorso è analogo se scegliessimo un'estrazione veloce, dovremmo "shiftare" verso destra gli elementi per lasciare spazio ad un nuovo elemento. Per non contare il fatto che gli Array sono una struttura dati statica.

Ecco, per quanto sembri impossibile trovare un'implementazione vantaggiosa delle code con Array, alcuni geniali informatici ci sono riusciti: questa viene detta "implementazione mediante Array circolari". Questa non verrà discussa nella dispensa, tuttavia si invita il lettore ad approfondire tale argomento.

4.5 Coda con priorità

È doveroso anche parlare di una particolare variante della struttura dati coda: la **coda con priorità**. La differenza principale tra le due strutture è che la posizione di un elemento qualsiasi è dettata **non** dal momento in cui è stato inserito, bensì dal valore di una determinata grandezza, detta appunto priorità, che misura "l'importanza" di un certo dato. Di conseguenza, gli elementi di una coda con priorità sono collocati in ordine crescente (o decrescente, a seconda dei casi) rispetto alla "priorità" che hanno.

Facciamo degli esempi di coda con priorità, anche in merito ad altre strutture dati:

- Prima di tutto, nella vita reale una coda di priorità può essere vista come la coda ospedaliera: se due persone hanno lo stesso codice, allora ha priorità quella che è entrata per prima; se tuttavia dovesse presentarsi una persona con un codice più alto (e dunque una priorità maggiore), questa verrebbe posizionata in modo da uscire PRIMA delle altre due.
- Possiamo vedere come coda con priorità anche un array ordinato, dove la grandezza "priorità" è rappresentata dal valore dell'elemento associato ad un determinato indice i. Essendo infatti ordinato, abbiamo che gli elementi con indice più basso saranno anche quelli col valore minore (priorità inferiore), mentre quelli con indice più alto saranno anche quelli coi valori maggiori, e dunque priorità più alta.

Un problema sostanziale delle code con priorità è il cosiddetto problema della **starvation** (o attesa illimitata): se continuano a subentrare solo elementi con priorità alta, quelli con priorità bassa non usciranno mai dalla coda.

4.6 La Pila (Stack)

La Pila è una struttura dati astratta che si ispira al concetto di una pila di oggetti: pensiamo ad una pila di piatti: l'estremità in cui inseriamo i piatti è quella più in alto, e il primo piatto ad essere tolto è sempre quello più in alto. Dunque ne possiamo dedurre che le operazioni di inserimento ed estrazione avvengono dalla stessa estremità. La Pila segue infatti il comportamento di *Last In First Out* (LIFO).

Un esempio tecnico di Pila risiede nella "Pila delle chiamate delle funzioni". Infatti, quando chiamiamo una `funzione1` da un programma principale, questa viene messa in una pila (chiamata formalmente *Call Stack* o Stack delle funzioni). Se ad esempio all'interno di `funzione1` avessimo la chiamata a `funzione2`, quest'ultima verrebbe subito eseguita, cioè verrebbe subito messa sullo Stack. Inoltre, quando sarà terminata l'esecuzione della `funzione2`, questa verrà rimossa dallo Stack (Ricorda: *Last In First Out* = l'ultima ad entrare è anche la prima ad uscire) e riprenderà l'esecuzione della `funzione1`.

Sulla Pila vengono definite due operazioni:

- Inserimento (*Push*);
- Estrazione (*Pop*).

Dato che le operazioni lavorano su un'unica estremità della Pila, per riferirci a questa in un algoritmo passiamo un solo puntatore chiamato `top`:

```
1       def pushArray(top, k, max_size, curr_size):
2       if (curr_size == max_size): return
3       array[top+1] = k
4       top++
5       curr_size++
```

Listing 4.7: Implementazione del Push tramite Array

```
1       def popArray(top, curr_size):
2       if (curr_size == 0): return
3
4       e = array[top]
5       array[top] = None
6       top -= 1
7       curr_size -= 1
```

```
8
9       return e
```

Listing 4.8: Implementazione del Pop tramite Array

```
1       def pushLista(top, k):
2       if (top == None):
3       top = k
4       return
5
6       k.next = top
7       top = k
8       return top
```

Listing 4.9: Implementazione del Push tramite Lista concatenata

```
1       def popLista(top):
2       if (top == None): return
3
4       e = top
5       top = top.next
6       e.next = None
7
8       return e, top
```

Listing 4.10: Implementazione del Pop tramite Lista concatenata

4.7 Albero

La struttura dati Albero è una struttura dati estremamente versatile, che abbiamo già utilizzato in passato (parlando di Alberi decisionali e di Heap), ma alla quale non abbiamo mai dato una definizione formale. Prima di perderci in formalismi, notiamo intanto che, come già visto, un albero in informatica può essere visto come un albero "vero" ma capovolto: la radice si trova in alto, mentre le foglie si trovano in basso.

Prima di poter dare la definizione formale di Albero è tuttavia necessario fornire una definizione di Grafo, l'ente matematico dal quale l'albero prende le sue proprietà.

Definizione 4.7.1: Grafo

Un Grafo è un ente matematico che indichiamo come $G = (V, E)$. Come si può vedere, ogni grafo è individuato da una coppia di insiemi:

- un insieme finito V che rappresenta i **nodi** (o vertici) del grafo;
- un insieme finito $E \subseteq V \times V$ degli **archi** (un arco è il collegamento tra due nodi). Come vediamo, gli elementi dell'insieme E saranno tutti coppie non ordinate di nodi, ciascuno appartenente a V.

Definiamo inoltre il concetto di **cammino**, cioè di una sequenza $(v_1, v_2, \ldots, v_k)$ di nodi distinti di V tale che $(v_i, v_{i+1}) \in E$, cioè due nodi adiacenti della sequenza sono collegati da un arco. Informalmente possiamo dire che è il percorso che si può tracciare da un nodo v_1 ad un altro nodo v_k. Se per ogni scelta di due vertici u, v abbiamo un cammino che li connette, diciamo che il grafo è **connesso** (il cammino non deve per forza avere lunghezza 1, basta che esista).

Inoltre, se ad esempio in un cammino del tipo $(v_1, v_2, \ldots, v_k)$ abbiamo un arco della forma (v_i, v_1) (con i qualsiasi, basta che l'arco sia connesso all'elemento di partenza della sequenza) diciamo che questo cammino è un **ciclo**. Anche qui informalmente possiamo dire che un ciclo è un cammino che inizia e finisce nello stesso punto. Se un grafo NON presenta alcun ciclo, si dice **aciclico**.

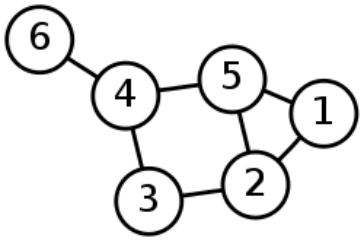


Figura 4.1: Rappresentazione visiva di un grafo.

Date le dovute nozioni per quanto riguarda i grafi, possiamo procedere con la definizione di albero.

Definizione 4.7.2: Albero

Un albero è semplicemente un grafo connesso e aciclico.

Lemma: *Se G è un grafo connesso e aciclico ed eliminiamo da G un QUALSIASI arco, G si suddivide in due "sottografi" G_1 e G_2, entrambi connessi e aciclici.*

Teorema: *Se G è un grafo, le due seguenti affermazioni sono equivalenti:*

1. *G è connesso e aciclico (in altre parole, G è un albero);*
2. *G è connesso e $|E| = |V| - 1$ (il numero di archi è uno in meno del numero di nodi).*

Da ciò ne segue che in un albero il numero di archi è sempre uno in meno del numero di nodi.

Un particolare sottoinsieme degli alberi è costituito dagli **Alberi radicati**, cioè alberi in cui ha particolare importanza un nodo detto "radice".

Gli alberi radicati godono di alcune proprietà:

- I nodi sono organizzati in **livelli**, numerati in ordine crescente a partire dalla radice (che per convenzione è considerata al livello 0).
- L'**altezza** di un albero radicato equivale alla lunghezza del più lungo cammino da una radice ad una foglia. Generalmente l'altezza di un albero si indica con h e questo vuole anche dire che possiede $h + 1$ livelli, di norma numerati da 0 ad h.
- Dato un nodo v (che non sia la radice) il primo e unico nodo che si incontra nel cammino che va da v alla radice è detto **"padre di v"**. Possiamo infatti dire che la radice è l'unico nodo che non ha un padre. In generale, tutti i nodi che si incontrano in un cammino che va da v alla radice si dicono **"antenati di v"**.
- I primi nodi che si incontrano in un cammino che va da v ad una qualsiasi foglia si dicono **"figli di v"**. Analogamente alla radice, le foglie sono gli unici nodi che non hanno figli. In generale, tutti i nodi che si incontrano in un cammino che va da v ad una qualsiasi foglia si dicono **"discendenti di v"**.
- Un albero radicato si dice **ordinato** se possiamo attribuire un qualche ordine ai figli di ciascun nodo (cioè ad esempio se scegliamo una qualche relazione per identificare il primo, secondo, terzo, ..., k-esimo figlio).

4.7.1 Alberi binari

Una particolare categoria di alberi radicati ordinati è rappresentata dai cosiddetti **Alberi binari**, cioè alberi radicati che hanno la proprietà di avere **al più** 2 **figli** per ciascun nodo. Poiché sono ordinati, e i figli sono al massimo due, possiamo distinguere tra figlio sinistro e figlio destro.

Forniamo ora una definizione di qualcosa che avevamo già visto prima mentre parlavamo dell'Heap e della sua rappresentazione concettuale:

- Un albero binario si dice **completo** se tutti i livelli contengono il massimo numero possibile di nodi.
- Un albero binario si dice **quasi completo** se risulta completo fino ad un fissato livello $h-1$, mentre al livello h è riempito completamente da sinistra a destra fino ad un certo punto.

Forniamo adesso dei dati **importantissimi** che riguardano gli alberi completi di altezza h. Se l'albero binario è completo, diciamo che il numero totale di nodi è n. In quel caso valgono le seguenti formule:

- **Altezza:** $h = \log_2(n)$.
- **Numero di foglie:** 2^h.
- **Numero di nodi interni:** calcolabile tramite la serie geometrica

$$\sum_{i=0}^{h-1} 2^i = \frac{2^h - 1}{2 - 1} = 2^h - 1$$

- **Numero totale di nodi (n):** $2^h + (2^h - 1) = 2^{h+1} - 1$.

4.7.2 Rappresentazione in Memoria degli Alberi

Abbiamo già parlato di come sia differente il discorso tra strutture dati astratte e concrete: l'albero non fa differenza, in quanto ciò di cui abbiamo discusso finora è puramente teorico, e si deve dunque trovare un modo di implementarlo fisicamente all'interno della memoria.

1. Rappresentazione tramite record e puntatori

In questo tipo di rappresentazione utilizziamo il concetto definito precedentemente con la Lista concatenata, con alcune piccole differenze. Ogni singolo nodo è costituito da un *record* che contiene al suo interno 3 campi:

- `key`: il "valore" del nodo;
- `left`: un puntatore che fa riferimento al record del figlio sinistro;
- `right`: un puntatore che fa riferimento al record del figlio destro.

Chiaramente `left` e `right` possono essere `null` in caso il nodo non abbia il corrispondente figlio.

Come per le liste puntate, l'accesso ai nodi dell'albero si fa a partire dalla radice, che sarà dunque il nodo fornito per ogni funzione che utilizza un albero. I **vantaggi** legati a questa memorizzazione sono gli stessi delle Liste concatenate: dinamicità della struttura, inserimento di nodi, spostamento di nodi... tutto fatto semplicemente cambiando dei riferimenti. Tuttavia sono numerosi anche gli **svantaggi**: non è consentito l'accesso diretto, dunque per accedere ai dati di un nodo bisogna partire dalla radice e scorrere tutto l'albero, spostandosi di padre in figlio. Inoltre, avendo due puntatori per ogni record, non è chiaro a priori se si debba andare a destra o sinistra.

2. Rappresentazione posizionale

In questo caso si usa come struttura dati concreta l'array, e il concetto è lo stesso utilizzato per memorizzare gli Heap: la radice occupa l'elemento dell'array con indice 0, e per ogni nodo ad un generico indice i vale che il figlio sinistro si trova in posizione $2i+1$ e il destro in posizione $2i+2$. Il padre di ogni nodo si calcolerà come $\lfloor \frac{i-1}{2} \rfloor$.

Per quanto alcune operazioni con questa rappresentazione risultino più rapide, lo svantaggio maggiore è dato da un **notevole spreco di memoria**: essendo l'array una struttura dati statica, bisogna allocare da principio un certo numero di elementi. Bisogna dunque conoscere a priori l'altezza dell'albero h, in modo da allocare $2^{h+1}-1$ elementi. TUTTAVIA questo è il numero dei nodi che sono presenti in un albero binario *completo*: se così non fosse, avremmo un numero di nodi esponenzialmente minore, con conseguente spazio allocato e lasciato vuoto.

3. Vettore dei padri

Per questa rappresentazione, facciamo uso di 2 Array:

- **Array delle chiavi (A):** un array in cui ogni elemento rappresenta il valore del nodo nell'albero;
- **Array dei padri (P):** un array con la stessa lunghezza del precedente, che però ad ogni elemento di indice i associa l'indice dell'elemento di A che è padre di $A[i]$.

Come le altre rappresentazioni tramite array, la radice si trova ad $A[0]$ e il corrispondente $P[0]$ ha un valore speciale, o nullo.

Confronto delle Operazioni in base alla Rappresentazione

Vedremo di seguito alcune operazioni che possiamo voler fare su un albero, analizzandone l'implementazione per ciascuna struttura:

- Trovare il padre di un nodo:

  - *Record e puntatori:* non abbiamo un modo rapido per farlo; non avendo un puntatore al padre, dovremo scorrere tutti i nodi. Tempo $\mathcal{O}(n)$.
  - *Rappresentazione posizionale:* calcolo aritmetico $\lfloor \frac{i-1}{2} \rfloor$. Tempo $\Theta(1)$.
  - *Vettore dei padri:* accesso diretto all'indice $P[i]$. Tempo $\Theta(1)$.

- Determinare se il nodo abbia 0, 1 o 2 figli:
  - *Record e puntatori:* si verifica il valore dei puntatori `left` e `right`. Tempo $\Theta(1)$.
  - *Rappresentazione posizionale:* si calcola il valore del figlio destro e sinistro e si controlla se l'indice ricade nei limiti o è nullo. Tempo $\Theta(1)$.
  - *Vettore dei padri:* avendo l'indice i, dobbiamo scorrere tutto l'array P per trovare i valori tali che $P[x] = i$. Tempo $\mathcal{O}(n)$.

- Determinare la distanza di un nodo dalla radice:
  - *Record e puntatori:* richiede di scorrere l'albero. Tempo $\mathcal{O}(h)$, con h altezza dell'albero.
  - *Rappresentazione posizionale:* calcolo aritmetico $\lfloor \log_2(i+1) \rfloor$. Tempo $\Theta(1)$.
  - *Vettore dei padri:* partiamo da $P[i]$ e risaliamo attraverso i puntatori $P[P[i]]$, $P[P[P[i]]]$, ... fino alla radice. Tempo $\mathcal{O}(h)$.

4.7.3 Visite degli Alberi

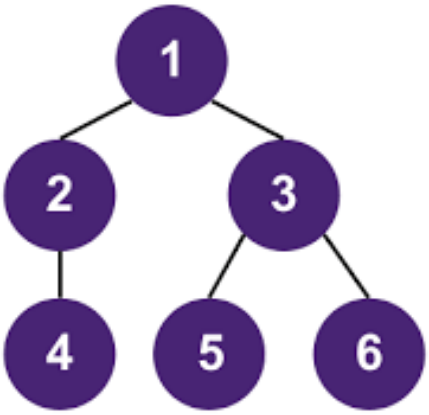
Problemi come quelli appena posti, ma anche altri, possono essere risolti definendo un'importante operazione sugli alberi: l'accesso "sequenziale" a tutti i nodi, uno dopo l'altro, viene detto **visita dell'albero**.

Siccome abbiamo già detto che gli alberi binari sono strutture ordinate, possiamo definire un ordine secondo il quale viene effettuata questa visita:

- **Visita in preordine** (*preorder*): viene visitato per prima il nodo, poi il suo sottoalbero sinistro e poi il suo sottoalbero destro (N - S - D);
- **Visita inordine** (*inorder*): viene visitato il sottoalbero sinistro del nodo, poi il nodo stesso, e infine il sottoalbero destro (S - N - D);
- **Visita in postordine** (*postorder*): viene visitato il sottoalbero sinistro del nodo, il sottoalbero destro e infine il nodo stesso (S - D - N).

Il termine “visitato” intende una qualsiasi operazione che può essere eseguita sul nodo stesso. Per semplicità, assumiamo che sia un’operazione eseguibile in tempo costante.

Se consideriamo il seguente albero binario, le diverse visite produrranno anche un risultato diverso:



Risultati delle Visite:

- **Preordine:** 1 - 2 - 4 - 3 - 5 - 6
- **Inordine:** 4 - 2 - 1 - 5 - 3 - 6
- **Postordine:** 4 - 2 - 5 - 6 - 3 - 1

Figura 4.2: Esempio di visite su un albero binario.

Vediamo ora il codice relativo alla visita di un albero.

```
    def visita_generica(root):
      if root is None: return

      # Operazioni su nodo corrente se pre-ordine
      visita_generica(root.left)
      # Operazioni su nodo corrente se in-ordine
      visita_generica(root.right)
      # Operazioni su nodo corrente se post-ordine
```

Listing 4.11: Algoritmo generico di visita di un albero

Come vediamo, il codice delle visite è sempre lo stesso; l'unica cosa che cambia è appunto il punto in cui vengono fatte le effettive operazioni (e questo dipende dalla visita che stiamo operando).

Il calcolo del costo computazionale è invece più complesso, in quanto non abbiamo una vera e propria garanzia sulla struttura dell'albero. Essendo una funzione ricorsiva, cerchiamo di impostare l'equazione di ricorrenza:

$$\begin{cases} T(n) = T(k) + T(n-k-1) + \Theta(1) \\ T(1) = \Theta(1) \end{cases}$$

Avendo un parametro k, l'unica speranza che abbiamo di risolverla è con il metodo di sostituzione, facendo dunque delle ipotesi sulla natura del parametro k:

Caso bilanciato: si verifica quando l'albero binario è completo, e dunque ogni chiamata ricorsiva divide in due sottoalberi esattamente uguali. In questo caso l'equazione di ricorrenza diventa $T(n) = 2T(n/2) + \Theta(1)$. In questo caso l'equazione ricade nel Caso 1 del *Master Theorem*, con soluzione $\Theta(n)$.

Caso sbilanciato: si verifica quando ogni chiamata ricorsiva divide in un sottoalbero di altezza 0 e l'altro altezza $n-1$ (è il caso in cui $k = 0$). In questo caso l'equazione diventa $T(n) = T(n-1) + \Theta(1)$, la cui soluzione sarà $\Theta(n)$.

Essendoci fatti delle idee più concrete riguardo la natura della possibile soluzione, se proviamo a risolvere con il metodo di sostituzione otteniamo che la soluzione è effettivamente $\Theta(n)$.

Questo vale anche se l'albero venisse memorizzato con notazione posizionale, mentre per la rappresentazione mediante vettore dei padri abbiamo un costo di $\mathcal{O}(n^2)$, dato che abbiamo due array che dobbiamo progressivamente scorrere.

4.7.4 Visite per livelli (Breadth-First Search)

E se invece volessimo visitare l'albero, memorizzato tramite record e puntatori, **livello per livello**? Si noti che si è specificata la memorizzazione tramite record e puntatori in quanto nella rappresentazione mediante un qualsiasi Array i dati sono già organizzati per livello, cambia solo il modo in cui accediamo i nodi, se dal basso verso l'alto o dall'alto verso il basso.

Per riuscire a risolvere questo problema facciamo uso di una **coda d'appoggio**, nella quale metteremo ogni nodo per poi toglierlo. Si ricorda la struttura della coda: il primo nodo ad entrare sarà anche il primo ad uscire. Se mettiamo come primo nodo la radice dell'albero, allora sequenzialmente verranno aggiunti il suo figlio sinistro e destro, poi i loro sottoalberi in maniera analoga.

Il codice per operare tale visita viene lasciato di seguito, con la dovuta spiegazione.

```
1   def visita_per_livelli(root, head, tail):
2     if root == None: return
3
4     enqueue(head, tail, root)
5     while !coda_vuota(head):
6       p = dequeue(head, tail)
7       # Operazioni che vogliamo fare sul nodo
8       if p.left != None:
9         enqueue(head, tail, p.left)
10      if p.right != None:
11        enqueue(head, tail, p.right)
```

Listing 4.12: Algoritmo di visita per livelli tramite Coda

Spiegazione algoritmo 4.7.1: Visita per livelli

- Come primissima cosa, aggiungiamo la radice dell'albero alla coda. Questo è un passaggio cruciale, in quanto l'accesso alla radice di un albero significa anche l'accesso a tutti i nodi sottostanti.

- Facciamo partire un ciclo che si ripete finché è presente almeno un elemento nella coda.
- Eseguiamo il `dequeue()` sull'elemento che si trova alla `tail` della coda. Ragioniamo sulla prima iterazione: a quel punto l'unico elemento nella coda sarà la radice. Stiamo quindi rimuovendo la radice. Tuttavia prendiamo un riferimento alla radice, quello che è `p` nel nostro codice. A quel punto abbiamo un riferimento al nodo dell'albero, e possiamo accedere ai dati relativi a quel nodo `p`, come i suoi nodi figli, se esistono.
- Eseguiamo l'operazione che vogliamo fare sul nodo, per esempio una stampa o l'incremento di un contatore.
- Se i figli esistono, li aggiungiamo da sinistra a destra alla coda, in modo che il prossimo nodo sul quale verrà eseguita la `dequeue()` sarà il figlio sinistro del nodo `p` che abbiamo appena controllato. Questo discorso si ripete finché non troviamo l'ultima foglia.

Il costo computazionale di questo algoritmo è chiaramente un $\Theta(n)$, in quanto deve essere passato in rassegna ogni nodo dell'albero, e l'inserimento e rimozione dalla coda sono operazioni che vengono eseguite in tempo costante.

4.8 Dizionario

Come ultima struttura dati astratta andiamo a vedere una struttura già nota dal linguaggio Python: i dizionari.

Definizione 4.8.1: Dizionario

Un dizionario è una struttura dati che permette di gestire un insieme dinamico di dati, che generalmente è un **insieme totalmente ordinato** (ossia un insieme in cui per ogni due elementi che prendiamo, sappiamo dire che relazione occorre tra loro. Ricorda le relazioni di ordine totale di Metodi!), sul quale si possono compiere le seguenti 3 operazioni:

- Inserimento di un elemento;
- Cancellazione di un elemento;
- Ricerca di un elemento.

Con le strutture dati precedenti abbiamo già visto come le seguenti operazioni fossero implementabili in modo abbastanza semplice mediante gli Array, le Liste concatenate semplici e le Liste concatenate doppie. Tuttavia, non sempre queste operazioni risultavano efficienti.

La richiesta per la struttura dati dizionario è dunque che sia in grado di implementare queste tre operazioni in maniera abbastanza efficiente. Concretamente, un dizionario può essere implementato nei seguenti 3 modi:

1. Tabella ad indirizzamento diretto;
2. Tabelle hash;
3. Alberi binari di ricerca.

4.8.1 Tabelle ad indirizzamento diretto

Questo primo metodo è abbastanza semplice: si crea un Array nel quale ogni indice corrisponde al valore della chiave dell'elemento da memorizzare in tale posizione. Dunque si stabilisce una corrispondenza biunivoca tra una chiave e l'indice dell'array a cui questa è associata. Ed è COSÌ biunivoca che è letteralmente lo stesso numero.

Se n è la grandezza dell'array e m sono le possibili chiavi che avremo, ci stiamo ponendo nell'ipotesi in cui $n \leq m$. Per quanto poi le operazioni di inserimento, ricerca e cancellazione siano efficienti (essendo la corrispondenza biunivoca, basta fare riferimento sempre ad `A[k]` per un dato elemento) e vengano eseguite in tempo costante $\Theta(1)$, ci rendiamo conto di un sostanziale limite tecnico: se volessimo che all'interno dell'Array, ad esempio, fossero salvate le matricole di ogni studente di un'Università, ci rendiamo conto di come dovremmo riservare MILIONI di locazioni di memoria, cioè una per ogni matricola di studente già esistente PIÙ quelle per i nuovi studenti che si iscriveranno in futuro (non possiamo impedire le iscrizioni solo perché abbiamo finito lo spazio in memoria).

Le cose quindi sono due:

- Dovremmo allocare un Array di tantissime locazioni, consumando tantissimo spazio in memoria.
- In caso non venissero riempite tutte le locazioni riservate, si verificherebbe un enorme **spreco di memoria** (Pensiamo ad esempio: mi alloco 1 milione di celle di memoria per altrettanti studenti, ma poi se ne iscrivono solo 1000. Lo spreco si commenta da sé).

Sono questi i motivi per cui questa non è quasi mai la rappresentazione fisica scelta per un dizionario, salvo che m non sia un numero davvero molto piccolo.

4.8.2 Tabelle hash

Quando l'insieme dei valori memorizzabili è molto più grande di quelli che potrebbero effettivamente essere memorizzati, esiste una soluzione detta **Tabella Hash**.

L'idea non è più quella di procedere per corrispondenza biunivoca, bensì di "associare" ad ogni chiave possibile un determinato indice, calcolato tramite un'apposita funzione detta appunto **"Funzione di Hash"**. Se chiamiamo questa funzione h, questa prenderà come input un qualsiasi valore dell'insieme di TUTTI I VALORI POSSIBILI (esempio: di tutti i numeri interi, o delle matricole) e restituirà un valore nell'intervallo $[0, m-1]$ che sono appunto gli indici disponibili per memorizzare un determinato record.

Per quanto questa sia una buona idea, e non crei la corrispondenza biunivoca di prima tra indice $\leftrightarrow$ chiave, abbiamo un problema di fondo: per il principio della piccionaia, dato che l'insieme di partenza è MOLTO più grande di quello di arrivo, non possiamo escludere che due chiavi risultino nello stesso indice. Tale situazione viene chiamata **collisione**. È proprio per questo che una buona funzione di hash deve rendere il più improbabile possibile che ciò accada, rendendo equiprobabile ogni valore nell'insieme $[0, m-1]$.

Una possibile soluzione per il problema delle collisioni sono le cosiddette **Liste di trabocco**: ad ogni singolo elemento dell'Array di m elementi corrisponde un puntatore ad una lista puntata. Se a due chiavi va a corrispondere lo stesso indice nell'Array, mettiamo quell'elemento in testa alla lista puntata corrispondente a quell'indice.

4.8.3 Alberi binari di ricerca (ABR)

Uno degli altri metodi per definire un dizionario è mediante gli **Alberi Binari di Ricerca** (ABR). Questi sono dei particolari alberi binari per i quali valgono le seguenti proprietà:

- Ogni nodo dell'albero contiene una chiave;
- Il valore della chiave contenuta nel figlio sinistro è **minore** del valore della chiave contenuta nel nodo padre;
- Il valore della chiave contenuta nel figlio destro è **maggiore** del valore della chiave contenuta nel nodo padre;

Ne risulta dunque che alla sinistra di un nodo ci saranno SOLO nodi con un valore minore di quello del nodo stesso, mentre alla sua destra ci saranno SOLO nodi con un valore maggiore di quello del nodo stesso.

Per gli ABR valgono tutte le operazioni definite per gli alberi, come ricerca di un elemento, minimo, massimo, predecessore, successore (operazioni di interrogazione), inserimento e cancellazione (operazioni di modifica).

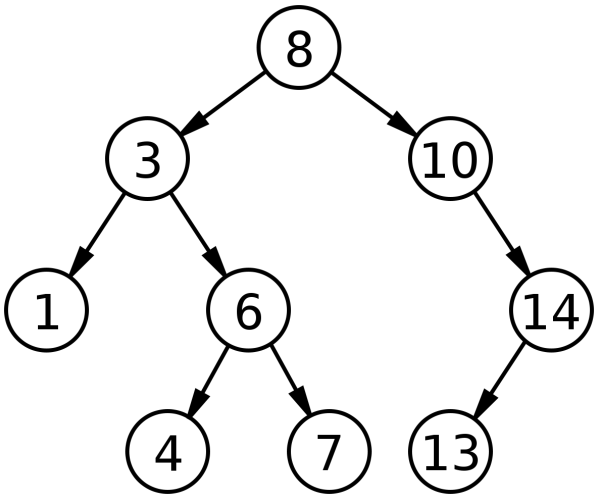


Figura 4.3: Rappresentazione di un Albero Binario di Ricerca (ABR).

Si noti una proprietà interessante: il **minimo** dell'ABR si trova nel nodo più a sinistra (non è detto che sia una foglia, ma comunque si trova tutto a sinistra); il **massimo** dell'ABR si trova nel nodo più a destra (anche qui non è detto che sia una foglia).

Grazie a questa proprietà, notiamo che eseguire una visita in-ordine sull'albero equivale a stampare i nodi dell'albero ordinati in **ordine crescente**. Un algoritmo di ordinamento su ABR è dunque articolato in due fasi:

1. Inserimento delle n chiavi da ordinare all'interno di un ABR inizialmente vuoto;
2. Visita in ordine dell'ABR appena costruito.

Vediamo adesso alcune operazioni sugli ABR.

Ricerca di un valore k

```
    def ricerca(root, k):
      if root == None: return None

      if root.key == k: return root
      if root.key < k:
        return ricerca(root.right, k)
      else:
        return ricerca(root.left, k)
```

Listing 4.13: Ricerca di un valore in un ABR

Come vediamo il concetto è simile a quello della ricerca binaria in un Array ordinato. Potremmo dunque pensare che il costo computazionale dell'algoritmo sia $\Theta(\log n)$. Questo potrebbe essere vero, se l'albero fosse completo o quasi completo. Tuttavia per gli ABR, al contrario ad esempio degli Heap, non abbiamo la garanzia che l'albero rispetti questa clausola: potrebbe essere completo, semi-completo, oppure anche sbilanciato da un lato.

Per identificare il costo computazionale di questa ricerca dobbiamo dunque esprimerlo in funzione dell'altezza dell'ABR, che indichiamo con h. Essendo poi un algoritmo ricorsivo, dobbiamo scriverne l'equazione di ricorrenza:

$$T(h) = \Theta(1) + T(h-1)$$

Perché "$h-1$"? Perché per ogni chiamata ricorsiva vengono effettuate operazioni in tempo costante e poi la chiamata ricorsiva viene effettuata su uno dei nodi che stanno direttamente sotto a quello appena considerato, che avrà altezza proprio $h-1$. Questo, ricorsivamente, ci fa ottenere che il costo computazionale di questo algoritmo è $\Theta(h)$.

Inserimento di un valore k

L'inserimento di un valore ovviamente punta ad essere molto semplice: si confronta il valore del nodo corrente con quello da inserire, se è più grande procediamo a destra e altrimenti a sinistra. Quando arriviamo ad un nodo con valore `None` possiamo inserire il nostro nodo con valore k. Nonostante sia un'operazione molto semplice, dobbiamo apportare una piccola modifica alla struttura di ogni singolo nodo dell'albero: oltre ai campi `key`, `left`, `right`, aggiungiamo anche un puntatore `parent` che, come si intuisce dal nome, punta al nodo padre di quel nodo.

```
1     def insert(root, k):
2       x, y = root, None
3       while (x != None):
4         y = x
5         if k.key < x.key:
6           x = x.left
7         else:
8           x = x.right
9       k.parent = y
10
11      if y == None:
12        root = k
13      else:
14        if k.key < y.key:
```

```
15            y.left = k
16          else:
17            y.right = k
18
19        return root
```

Listing 4.14: Inserimento di un valore in un ABR

Spiegazione algoritmo 4.8.1: Inserimento in ABR

- Definiamo delle variabili d'appoggio: `x` che terrà traccia del nodo che stiamo considerando in questo momento; `y` che terrà traccia del PADRE di quel nodo.
- Vogliamo andare avanti a cercare la posizione giusta finché il nodo che stiamo considerando non è nullo, perché in tal caso potremmo fermarci e mettere il valore a quel nodo lì, assicurandoci che il padre del nodo sia impostato correttamente.
- Siccome dovremo cambiare il valore di `x` per permettergli di spostarsi nel sottoalbero destro o sinistro, assegniamo `y = x` per assicurarci di mantenere un riferimento al padre del nodo in cui siamo scesi.
- Controlliamo poi come abbiamo fatto nella ricerca binaria: se il valore che dobbiamo inserire è più grande del nodo corrente, allora scendiamo a destra (`x = x.right`); altrimenti scendiamo a sinistra (`x = x.left`).
- All'ultima iterazione avremo che `x` risulta `None`. Fortunatamente abbiamo tenuto un riferimento al padre dell'ultimo nodo `x`, che è proprio `y`. Possiamo dunque impostare `k.parent = y` (quello che precedentemente era il padre dell'ultimo nodo `x`).
- Adesso: se `y == None` vuol dire che oltre alla radice non c'era nulla, dunque impostiamo come nuova radice il nodo che vogliamo inserire.
- Altrimenti, avendo che adesso `y` è il padre di `k`, dobbiamo vedere dove mettere questo nodo `k`, se a destra o sinistra di `y`. Per questa assegnazione seguiamo la solita logica di organizzazione dei nodi in un ABR.

Il costo computazionale di questo algoritmo è un $\mathcal{O}(h)$, dato che il ciclo `while` viene eseguito al più h volte (non è detto che il nodo vada messo su una delle foglie all'ultimo livello).

Ricerca di minimo e massimo

Come già detto in precedenza, la struttura dell'ABR è tale da permetterci di dire che il valore minimo si trova tutto a sinistra, mentre il massimo tutto a destra. Ciò significa che per ciascuna delle ricerche dovremo scorrere un singolo cammino. Intuitivamente, il costo computazionale sarà un $\Theta(h)$, cioè proporzionale all'altezza dell'ABR.

Ricerca di predecessore e successore

Prima di procedere con l'analisi dell'algoritmo, ricordiamo il significato di predecessore e successore. Con **predecessore** si intende l'elemento che precederebbe un dato elemento k se le chiavi fossero ordinate; con **successore** si intende, analogamente, l'elemento che seguirebbe un dato elemento k se le chiavi fossero ordinate. Dunque, la ricerca di predecessore o successore non ha affatto legame con la struttura padre-figlio dell'ABR.

Caso 1: Supponiamo che vogliamo trovare il predecessore di un nodo che **ha** un sottoalbero sinistro (come 20 nel precedente esempio). In tal caso, il predecessore di tale nodo sarà il nodo più a destra del sottoalbero sinistro. Questo avviene intuitivamente per la struttura dell'ABR: dato che stiamo cercando il predecessore, dovremo andare nel sottoalbero sinistro (che contiene i numeri più piccoli); a quel punto, dovremo prendere il più grande dei valori in quel sottoalbero che, sempre per la struttura di un ABR, si trova tutto a destra. Basta quindi fare una ricerca del massimo che parte dal figlio sinistro del nodo di cui vogliamo cercare il predecessore.

Caso 2: Se invece volessimo trovare il predecessore di un nodo che **non ha** sottoalbero sinistro, ciò vuol dire che quello sarà il nodo più a sinistra di un certo sottoalbero. Dunque il valore più piccolo di quel sottoalbero. Siccome si trova in un sottoalbero sinistro, per trovare il predecessore dovremo "salire" lungo i suoi antenati, finché non troviamo quello più piccolo di lui.

Per la ricerca del successore il ragionamento è analogo e simmetrico: se ci troviamo nel Caso 1 dobbiamo scendere di un livello lungo il sottoalbero destro, e poi tutto a sinistra; se invece ci troviamo nel Caso 2, significa che il nodo non ha un figlio destro. In tal caso dovremo salire tutti gli antenati verso sinistra e poi infine, quando possibile, verso destra.

Eliminazione di un elemento k

Il problema di eliminazione di un elemento da un ABR è forse il più complicato, poiché in caso volessimo eliminare un nodo che ha entrambi i figli dovremo aggiustare la struttura dell'ABR per evitare che quello originale si scomponga in due ABR più piccoli. Dunque, il nodo da mettere al posto di quello eliminato può essere solamente il predecessore o il successore del nodo da eliminare.

Analizzando infine i casi appena presentati:

1. Se il nodo da eliminare è una **foglia**, basta rintracciare quel nodo e porre a `None` il campo `parent`.
2. Se il nodo ha **un solo figlio**, si elimina il collegamento di quel nodo con l'ABR, e si pone il campo `parent` di suo figlio a puntare al SUO campo `parent`.
3. Se il nodo ha **entrambi i figli**, questo viene sostituito col suo predecessore/successore, che va quindi rimosso dalla sua posizione originale. Si noti inoltre che per trovare il predecessore/successore del nodo da eliminare si userà sicuramente il Caso 1 di quelli descritti precedentemente. L'eliminazione avverrà poi secondo il punto 1 o 2 qui sopra.

4.9 Alberi AVL

Avendo definito così bene la struttura dati degli ABR, che esegue ogni operazione in tempo proporzionale all'altezza dell'albero ($\Theta(h)$), è davvero necessario definire una nuova struttura ad albero? È possibile fare di meglio?

Domanda lecita, che ha una risposta: **sì**. Questo perché, come già detto, non abbiamo alcuna garanzia riguardo l'altezza dell'ABR: potrebbe essere logaritmica, come potrebbe essere lineare. In questo ultimo caso, tanto varrebbe utilizzare un comune array.

Siccome sappiamo che per un albero con n nodi l'altezza minima possibile è $\log_2 n$, dobbiamo trovare una strategia per ridurre l'altezza dell'albero: è così che nasce il concetto di **bilanciamento** di un albero. Un albero si dice bilanciato quando la sua altezza è $\Theta(\log n)$.

Per mantenere le operazioni definite prima in tempo logaritmico (se l'albero è bilanciato, $h = \log_2 n$) dobbiamo trovare un modo per bilanciarlo in tempo costante. Ed è proprio così che nascono gli alberi di ricerca **AVL**.

Prima di definire cosa è un AVL, introduciamo un altro concetto strettamente legato a questa nuova classe di alberi binari di ricerca. Associamo ad ogni nodo dell'albero un parametro detto **"fattore di bilanciamento"** (`fdb`). Questo fattore di bilanciamento è un valore numerico che si calcola nel seguente modo:

$$\texttt{nodo.fdb} = \texttt{altezza(nodo.left)} - \texttt{altezza(nodo.right)}$$

cioè la differenza di altezze tra il sottoalbero sinistro e il destro. Intuitivamente, una foglia ha $\texttt{fdb} = 0$, in quanto non ha né un sottoalbero sinistro né un sottoalbero destro.

Definizione 4.9.1: Albero AVL

Un albero AVL è un particolare albero binario di ricerca in cui il fattore di bilanciamento di **OGNI** nodo è un valore compreso tra -1 e $+1$.

Si è introdotto il concetto di "bilanciamento" proprio perché vogliamo trovare delle operazioni eseguibili in tempo costante che rendano l'albero bilanciato. Questo accade perché, in seguito ad operazioni di inserimento o cancellazione all'interno di un ABR, questo potrebbe sbilanciarsi.

Per ri-bilanciare l'albero, entrano in gioco delle particolari operazioni dette **"operazioni di rotazione"**: rotazione destra e rotazione sinistra. Prima di spiegare come avvengono queste rotazioni, vediamo un esempio grafico per rendere il concetto e per vedere come le proprietà dell'ABR rimangano rispettate.

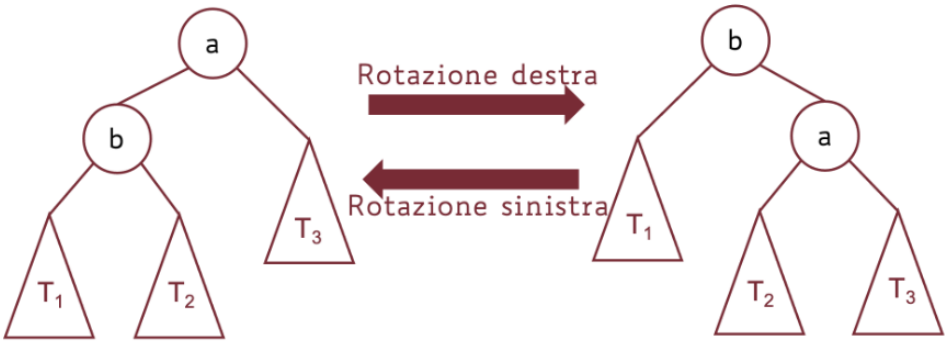


Figura 4.4: Rappresentazione delle operazioni di rotazione in un albero AVL.

Analizziamo solo la rotazione destra (per la sinistra il discorso è analogo):

- Inizialmente, la radice sarà il nodo a. Essendo a la radice ed essendo l'albero un ABR, sappiamo che tutti i nodi del sottoalbero sinistro saranno $< a$; analogamente, tutti i nodi del sottoalbero destro saranno $> a$.
- Tra tutti i nodi avremo dunque la seguente catena di disuguaglianze: $T_1 < b < T_2 < a < T_3$.

Vediamo che quando effettuiamo la rotazione destra, tutte queste proprietà rimangono rispettate:

- La nuova radice sarà b. Come vediamo, per la catena di disuguaglianze di prima abbiamo che $b < a$, quindi dovremo mettere necessariamente a nel sottoalbero destro di b.

- Dobbiamo poi sistemare i sottoalberi: l'unico sottoalbero più grande di a e dunque più grande di tutti è T_3, che infatti va alla destra di a; il sottoalbero T_2 è più grande di b (quindi va alla sua destra) ma più piccolo di a (quindi va alla sua sinistra), e vediamo come sia rispettato; infine, il sottoalbero più piccolo di tutti è T_1, che giustamente si trova a sinistra di b.

Come possiamo vedere, le due operazioni di rotazione destra e sinistra sono simmetriche: se le applichiamo una dopo l'altra otteniamo l'albero originale.

Notiamo come la struttura di un albero dopo una rotazione sinistra preveda uno "sbilanciamento verso sinistra" (cioè il sottoalbero sinistro diventa più alto di 1) mentre quella destra preveda uno "sbilanciamento verso destra" (cioè il sottoalbero destro diventa più alto di 1). Per questo bisogna stare attenti alla rotazione da applicare, perché potremmo rischiare di sbilanciare l'albero ancora di più.

Osserviamo, infine, come entrambe le rotazioni si basino solo sul concetto di "modificare i riferimenti tra i nodi", dunque il costo computazionale è chiaramente costante $\Theta(1)$.

4.9.1 Casistica delle Rotazioni

Avendo definito queste operazioni di bilanciamento, prima di procedere con le operazioni di inserimento e cancellazione vediamo QUANDO e QUALE rotazione usare in base alla casistica.

Rispondiamo alla prima domanda: **quando usare la rotazione?** Banalmente, quando abbiamo un nodo il cui fattore di bilanciamento NON è compreso nell'intervallo $[-1, 1]$. Ciò vorrà dire che avremo uno sbilanciamento destro/sinistro e dunque dovremo operare la rotazione opportuna.

Per quanto riguarda la seconda domanda, cioè **quale rotazione usare**, dobbiamo distinguere in 4 diversi casi per un dato nodo Z:

Caso 1: $\texttt{FDB}(Z) = +2$ e $\texttt{FDB}(Z.\texttt{left}) \geq 0$.
In questo caso significa che il sottoalbero sinistro del nodo Z che stiamo considerando ha un'altezza pari a 2 unità in più rispetto al sottoalbero destro. Utilizziamo una **singola rotazione destra** sul nodo Z, in modo da spostare uno dei due sottoalberi sinistri a destra e ottenere un fattore di bilanciamento nel range.

Caso 2: $\texttt{FDB}(Z) = -2$ e $\texttt{FDB}(Z.\texttt{right}) \leq 0$.
Questo è il caso speculare rispetto a prima, in cui il nodo Z ha un sottoalbero destro che è di 2 unità più profondo del sottoalbero sinistro. Il tutto si risolve con una **singola rotazione sinistra** sul nodo Z.

Caso 3: $\texttt{FDB}(Z) = +2$ e $\texttt{FDB}(Z.\texttt{left}) = -1$.

In questo caso se applicassimo solo una rotazione destra su Z, rischieremmo di sbilanciare l'albero in quanto il figlio sinistro ha $\texttt{FDB} = -1$. Dunque, per garantire un bilanciamento anche futuro, dobbiamo prima fare una **rotazione sinistra su** $Z.\texttt{left}$, e poi procedere con una **rotazione destra su** Z. Possiamo graficamente pensare a questo caso come a un nodo che ha un sottoalbero fatto come un "ginocchio piegato verso sinistra" ($<$).

Caso 4: $\texttt{FDB}(Z) = -2$ e $\texttt{FDB}(Z.\texttt{right}) = +1$.

Caso simmetrico al precedente, dove possiamo vedere la struttura di Z e dei suoi sottoalberi come un "ginocchio piegato verso destra" ($>$). Le operazioni da fare saranno simmetriche: prima applichiamo una **rotazione destra su** $Z.\texttt{right}$ e poi, una volta che il figlio destro sarà bilanciato, possiamo fare una **rotazione sinistra su** Z.

Avendo distinto nei vari casi in cui usare le rotazioni, possiamo procedere ad analizzare le operazioni di inserimento e cancellazione.

4.9.2 Inserimento e Cancellazione

Inserimento

L'inserimento di una chiave in un AVL avviene esattamente come l'inserimento all'interno di un ABR, solo che poi dobbiamo garantire che l'albero resti bilanciato, facendo dunque un controllo sui fattori di bilanciamento di ogni nodo. Notiamo però che, dato che l'inserimento opera su un singolo cammino dell'albero, dovremo controllare il fattore di bilanciamento SOLO dei nodi presenti su quel cammino, non su tutto l'albero.

Dunque, dopo aver inserito il nodo, dobbiamo controllare il fattore di bilanciamento di tutti i nodi lungo il cammino percorso (risalendo verso la radice): se troviamo un valore fuori dal range di bilanciamento ($+2$ o -2), controlliamo anche i nodi figli per verificare in quale dei 4 casi precedenti ci troviamo e applichiamo le dovute rotazioni.

Cancellazione

Come per l'inserimento, anche la cancellazione è analoga: segue le stesse regole della cancellazione in un ABR, ma ovviamente bisogna poi verificare che il bilanciamento dell'albero non sia stato alterato, risalendo il cammino e applicando ove necessario le rotazioni destre e sinistre descritte nei 4 casi.

4.10 B-Alberi

Abbiamo definito gli alberi AVL proprio per migliorare la struttura degli ABR e garantire che ogni operazione possa essere eseguita in tempo logaritmico. Tuttavia, questi presentano ancora alcune limitazioni che emergono in presenza di dati di grandi dimensioni: siccome ogni nodo dell'albero deve memorizzare una chiave e 3 puntatori (padre, figlio sinistro e figlio destro), non è detto che ci sia abbastanza spazio in memoria centrale per memorizzare tutti i nodi.

In tal caso, alcuni nodi andrebbero memorizzati su memoria secondaria (disco). Ma così facendo, non lavoreremmo più sulla memoria centrale (dove le operazioni di lettura e scrittura hanno costo costante), bensì su memoria secondaria, dove questo non è affatto vero! Dunque, il costo computazionale delle operazioni non sarebbe più strettamente logaritmico in termini di tempo di esecuzione, ma crescerebbe a causa dei pesanti accessi al disco.

Per ovviare a questo problema, sono stati progettati i **B-Alberi**, che sono la struttura dati adatta per gestire grandi quantità di dati memorizzati in memoria secondaria.

La maggiore peculiarità di questa struttura dati è il fatto che ogni nodo memorizza al suo interno più di una chiave, in particolare un **array di chiavi**: se non possiamo migliorare il costo delle singole operazioni di lettura e scrittura su memoria secondaria, tanto vale ridurre il numero di volte che queste vengono eseguite. Se queste vengono eseguite un numero di volte proporzionale all'altezza dell'albero, la soluzione è *ridurre l'altezza*. L'accorpamento di più chiavi in un singolo nodo riduce drasticamente l'altezza dell'albero.

Inoltre, un'altra proprietà dei B-Alberi è che mantengono il concetto di bilanciamento, garantendo dunque che l'altezza sia logaritmica rispetto al numero di nodi (**Attenzione!** Anche prima era logaritmica rispetto al numero di nodi, ma lo era anche rispetto al numero di chiavi, dato che il numero di chiavi coincideva col numero di nodi. Adesso, invece, il numero di nodi è enormemente inferiore al numero di chiavi).

Definizione 4.10.1: B-Albero

Un B-Albero di grado minimo $t \geq 2$ ha le seguenti proprietà:

- Il nodo radice contiene un numero s di chiavi, tutte memorizzate in ordine crescente, con $0 \leq s \leq 2t - 1$.
- Ogni nodo diverso dalla radice (sia interno che foglia) contiene un numero di chiavi s memorizzate in ordine crescente, tale che $t - 1 \leq s \leq 2t - 1$.

- Un nodo interno che contiene s chiavi, del tipo $k_1 < k_2 < \cdots < k_s$, ha esattamente $s+1$ figli. Se chiamiamo questi figli $T_1, T_2, \ldots, T_{s+1}$ vale che:
  - Le chiavi contenute nel figlio T_1 sono tutte $< k_1$.
  - Le chiavi contenute nel figlio T_2 sono tutte comprese tra k_1 e k_2 ($k_1 < T_2 < k_2$).
  - ...
  - Le chiavi contenute nel figlio T_i sono tutte comprese tra k_{i-1} e k_i ($k_{i-1} < T_i < k_i$).
- I nodi foglia si trovano tutti alla stessa profondità.

Figura 4.5: Rappresentazione di un B-Albero.

Come vediamo dall'esempio, se avessimo scelto di procedere con una chiave per ogni nodo, avremmo avuto 29 nodi diversi. Dunque l'altezza (supponendo un AVL perfettamente bilanciato) sarebbe stata $\approx \log_2(29)$, cioè circa 5. In questo caso, accorpando più chiavi, l'altezza è pari a 2, meno della metà! Questa differenza tende a crescere esponenzialmente all'aumentare del numero di chiavi.

Si può dimostrare come l'altezza massima di un B-Albero sia proporzionale a $\log_t(n)$. Più è grande la base del logaritmo t, minore sarà il suo valore complessivo, mantenendo l'altezza estremamente ridotta.

4.10.1 Rappresentazione in Memoria

Ricordiamo che ad ogni coppia di chiavi k_i e k_j di un nodo corrisponde un nodo figlio T_j limitato da esse. Questo vuol dire che dovremo avere un Array per salvare le chiavi del nodo e un Array per memorizzare i puntatori ai vari figli.

La dimensione dell'array delle chiavi sarà s, mentre la dimensione dell'array dei puntatori sarà $s+1$. Considerando gli intervalli, per accedere a un dato figlio basterà prendere l'indice della chiave che lo "limita superiormente":

- Il primo figlio $T_0 < k_0 \rightarrow$ indice 0 dell'array dei figli.
- Il secondo figlio $k_0 < T_1 < k_1 \rightarrow$ indice 1.
- L'ultimo figlio $T_{s+1} > k_s \rightarrow$ indice s dell'array dei figli (non $s+1$, perché si parte da 0).

4.10.2 Operazioni sui B-Alberi

Ricerca

La ricerca segue lo stesso principio degli ABR, ma in ogni nodo, anziché fare un singolo confronto, dobbiamo cercare la chiave all'interno dell'Array associato. Fortunatamente l'array è ordinato, dunque possiamo usare la **ricerca binaria**. L'operazione si articola così:

1. Partiamo dalla radice ed eseguiamo la ricerca binaria nell'array delle chiavi.
2. Se troviamo k, restituiamo l'indice e un riferimento al nodo.
3. Se NON lo troviamo, la ricerca binaria ci indicherà la posizione logica in cui k dovrebbe trovarsi. Usiamo quell'indice per seguire il puntatore al figlio corrispondente e ripetiamo il processo.

Il costo computazionale in termini di accessi al disco (che è la metrica principale nei B-Alberi) è pari all'altezza dell'albero, ovvero $\mathcal{O}(\log_t n)$, che asintoticamente indichiamo con $\mathcal{O}(\log n)$.

Inserimento

L'operazione di inserimento prevede la discesa nell'albero per trovare la foglia corretta in cui inserire la chiave. Anche in questo caso il costo computazionale asintotico è $\mathcal{O}(\log n)$. Tuttavia c'è un passaggio fondamentale: il limite di chiavi per nodo è $2t-1$. Cosa succede se l'inserimento sfora questo limite (raggiungendo $2t$)?

Si applica un'operazione di **Split** (divisione):

- L'array di $2t$ chiavi viene diviso in due array di dimensione t.
- Essendoci un nodo in più, serve un puntatore in più dal padre. La chiave mediana viene quindi "promossa" e spostata nel nodo padre.
- Se anche il padre è pieno, lo *split* si propaga verso l'alto, potendo arrivare fino alla radice (l'unico caso in cui l'altezza del B-Albero aumenta).

Cancellazione

Il problema principale della cancellazione non è tanto la rimozione della chiave in sé, ma il rischio che il nodo scenda sotto il limite inferiore di $t-1$ chiavi. Se questo accade, la proprietà si ristabilisce in due modi:

- **Spostamento (Rotazione):** Se un nodo fratello adiacente ha almeno t chiavi, si "prende in prestito" una chiave da lui (passando per il padre).
- **Fusione (Merge):** Se tutti i fratelli adiacenti hanno esattamente $t-1$ chiavi, si fondono due nodi fratelli assieme alla chiave separatrice del padre, creando un nodo da $2t-2$ chiavi.

Anche il *merge* può causare una carenza di chiavi nel padre, propagando la fusione verso l'alto e potenzialmente riducendo l'altezza dell'albero. Il costo asintotico globale rimane $\mathcal{O}(\log n)$.

4.11 Riepilogo Costi e Casistiche d'Uso

Struttura dati	**Ricerca**	**Min/Max**	**Pred/Succ**	**Insert**	**Delete**
Array disordinato	$\mathcal{O}(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Array ordinato	$\mathcal{O}(\log n)$	$\Theta(1)$	$\Theta(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Lista semplice	$\mathcal{O}(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\mathcal{O}(n)$
Lista doppia	$\mathcal{O}(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Coda	N/A	N/A	N/A	$\Theta(1)$	$\Theta(1)$
Pila	N/A	N/A	N/A	$\Theta(1)$	$\Theta(1)$
ABR	$\mathcal{O}(h)$	$\mathcal{O}(h)$	$\mathcal{O}(h)$	$\mathcal{O}(h)$	$\mathcal{O}(h)$
AVL	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
B-Albero	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Vediamo dunque di seguito le casistiche per l'uso di ciascuna struttura dati:

- **Array disordinato:** quando dobbiamo fare **tanti inserimenti e tante cancellazioni**, e dobbiamo gestire un **insieme di dati statico** in cui **l'ordine non conta**.
- **Array ordinato:** quando dobbiamo fare **tante ricerche di valori arbitrari o di valori minimi e massimi** di un **insieme statico** in cui **l'ordine conta**.

- **Lista concatenata semplice e doppia:** si usano quando non sappiamo a priori di quanti elementi è costituito l'insieme, dunque si usano quando abbiamo **tanti inserimenti e cancellazioni e non abbiamo un limite fissato al numero totale dei dati**.
- **Coda:** si usa quando vogliamo **gestire un insieme di dati in modalità FIFO** (*First In First Out*).
- **Pila:** si usa quando vogliamo **gestire un insieme di dati in modalità LIFO** (*Last In First Out*).
- **AVL:** Quando vogliamo un **costo bilanciato** (ma comunque molto efficiente) **per tutte le operazioni**. Tuttavia è conveniente **usarli quando stiamo operando con quantità di dati che non richiedono di essere memorizzate in memoria secondaria**.
- **B-Alberi:** Quando vogliamo un **costo bilanciato per tutte le operazioni** e, più importante, **prevediamo di avere una quantità di dati molto grande**, tale da essere memorizzata interamente in memoria secondaria.